Abstract

The Smart Home Energy Advisor is an AI-powered Model Context Protocol (MCP) server that transforms smart meter data into actionable energy intelligence. The system enables homeowners to ask energy-related questions in plain English and receive data-driven recommendations for cost optimisation, anomaly detection, and appliance scheduling. It uses a lightweight TypeScript and Node.js architecture with an in-memory data store, tariff-aware analysis, and pure analysis functions. The server exposes eight tools and a conversational energy_advisor prompt to MCP-compatible AI hosts, allowing an agent to select and chain tools autonomously. The project also includes a standalone browser dashboard and realistic synthetic meter data for demonstration. Its design emphasises tariff-aware savings, three-strategy anomaly detection, real-time meter-reading ingestion, and a zero-cloud infrastructure footprint.

Project Repository

GitHub Repository:

Table of Contents

List of Abbreviations

AI — Artificial Intelligence

CLI — Command-Line Interface

CRUD — Create, Read, Update, Delete

ESM — ECMAScript Modules

HTTP — Hypertext Transfer Protocol

IDE — Integrated Development Environment

IoT — Internet of Things

JSON — JavaScript Object Notation

JSON-RPC — JSON Remote Procedure Call

LLM — Large Language Model

MCP — Model Context Protocol

RAM — Random Access Memory

REST — Representational State Transfer

SSE — Server-Sent Events

TOU — Time-of-Use

UTC — Coordinated Universal Time

PWA — Progressive Web App

API — Application Programming Interface

ML — Machine Learning

SCADA — Supervisory Control and Data Acquisition

List of Figures

List of Tables

1. Executive Summary

The Smart Home Energy Advisor is an AI-powered MCP (Model Context Protocol) server that transforms raw smart meter data into actionable energy intelligence. It enables homeowners to ask energy questions in plain English and receive expert, data-driven recommendations — covering cost optimisation, anomaly detection, and appliance scheduling — with zero cloud infrastructure required.

The project bridges the gap between data-rich smart meters and insight-poor energy consumers by connecting directly to an AI agent through the MCP protocol, making utility-grade analytics accessible to every household via a single lightweight Node.js process.

2. Problem Statement

2.1 Rising and Unpredictable Energy Costs

Electricity prices have become increasingly volatile due to global energy market pressures, seasonal demand swings, and the transition to time-of-use and dynamic tariff structures. Households face bills that vary dramatically month-to-month with no clear indication of what drove the change or what corrective action is available.

2.2 Data Without Actionable Insight

Modern smart meters generate thousands of hourly readings. Existing consumer applications surface this data as charts and tables but cannot explain it, contextualise it against tariff schedules, or translate it into specific recommendations. The homeowner sees that their bill is high but does not know why or what to do about it.

2.3 Missed Tariff Optimisation

Shiftable appliances such as EV chargers, washing machines, and dishwashers frequently run during peak tariff hours. A single EV charger running at peak hours instead of overnight can cost a household hundreds of pounds per year in avoidable charges. No consumer tool currently bridges smart meter readings with tariff-aware scheduling recommendations at the appliance level.

2.4 Invisible Consumption Anomalies

Appliance faults, overnight power drains, and unusual consumption spikes go undetected until the monthly bill arrives. By that point, days or weeks of wasted energy cannot be recovered. Intra-day anomaly detection of this type is typically only available in industrial SCADA and building management systems, not household tools.

2.5 Complexity of AI Integration

Most smart home energy platforms expose REST APIs that require significant developer effort to integrate with AI systems. There is no standard, self-describing interface that allows an AI

agent to autonomously discover energy capabilities, select the right analysis tool, and reason over results — without manual orchestration code.

3.1 Conversational AI Interface

The energy_advisor MCP prompt pre-loads live meter context (current usage summary and any active anomalies) into the AI's system context, enabling open-ended natural language dialogue. Users ask questions like "Why is my bill so high?" or "Should I charge my EV now?" and receive expert, data-grounded answers without touching any UI.

3.2 Tariff-Aware Cost Optimisation

The server models three tariff structures — flat rate, time-of-use (TOU), and dynamic agile pricing — and computes exact per-appliance, per-cycle savings when shifted to cheaper hours. Recommendations include projected annual savings expressed in the user's currency, making financial impact immediately clear.

3.3 Three-Strategy Anomaly Detection

The server implements three distinct detection algorithms over the last 7 days of readings: spike detection (single reading exceeding 2x the rolling average), sustained-high detection (three or more consecutive hours above threshold), and overnight drain detection (unexpected load between 11 PM and 5 AM). Each anomaly is classified with a warning or critical severity.

3.4 Real-Time Data Ingestion

The add_meter_reading tool accepts live hourly readings pushed from any smart meter, IoT sensor, or home automation system. Analysis updates instantly on ingestion, enabling a reactive loop where an AI agent can detect and alert on anomalies in near real time without any polling infrastructure.

3.5 Zero Infrastructure Footprint

The entire server runs as a single Node.js process communicating over stdio using the JSON-RPC 2.0 protocol. There is no database, no web server, no cloud service, and no subscription required. The process consumes approximately 5 MB of RAM and is deployable on a Raspberry Pi sitting next to the smart meter.

Smart Home Energy Advisor — MCP Server Architecture

Overview

This is a Model Context Protocol (MCP) server built with TypeScript + Node.js that acts as an AI-powered personal electricity manager. It exposes 8 tools and 1 conversational prompt to any MCP-compatible AI host (like Bob).

Project Layout

smart-home-energy-advisor/

```
├── src/
│   ├── index.ts                ← MCP server entry point — registers all tools & prompt
│   ├── data/
│   │   ├── meterStore.ts       ← In-memory data store (meters, readings, appliances)
│   │   └── tariffs.ts          ← Tariff definitions & rate lookup
│   └── analysis/
│       └── energyAnalysis.ts   ← Pure analysis engine (no I/O side-effects)
├── dashboard.html              ← Standalone browser UI (chart dashboard)
├── index.mjs                   ← Pre-built ESM entry point
├── package.json
└── tsconfig.json
```

Layered Architecture

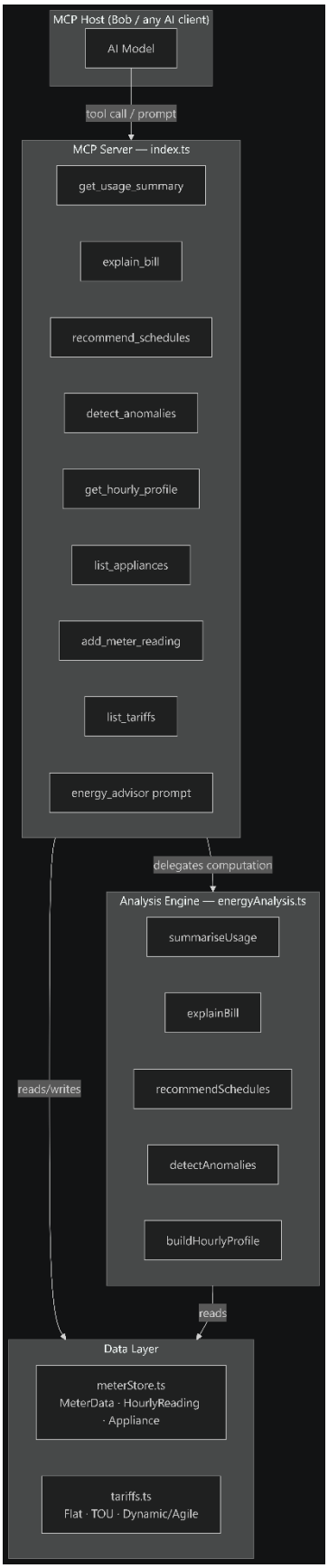

MCP Host (Bob / any AI client)
AI Model
tool call / prompt
MCP Server — index.ts
get_usage_summary
explain_bill
recommend_schedules
detect_anomalies
get_hourly_profile
list_appliances
add_meter_reading
list_tariffs
energy_advisor prompt
delegates computation
Analysis Engine — energyAnalysis.ts
summariseUsage
explainBill
recommendSchedules
detectAnomalies
buildHourlyProfile
reads/writes
reads
Data Layer
meterStore.ts
MeterData · HourlyReading
· Appliance
tariffs.ts
Flat · TOU · Dynamic/Agile

Figure 1: Smart Home Energy Advisor MCP Server layered architecture

Layer-by-Layer Breakdown

1. Entry Point — src/index.ts

- Instantiates McpServer from @modelcontextprotocol/server
- Registers **8** **tools** and **1** **prompt** using server.registerTool() / server.registerPrompt()
- Each tool uses **Zod schemas** for parameter validation
- All tools default to METER-DEMO-001 so the server works out-of-the-box
- Starts with StdioServerTransport — communication over stdin/stdout

Table 1: MCP tools and handler calls

Tool	Handler calls
get_usage_summary	summariseUsage()
explain_bill	explainBill()
recommend_schedules	recommendSchedules()
detect_anomalies	detectAnomalies()
get_hourly_profile	buildHourlyProfile()
list_appliances	getMeter() (direct)
add_meter_reading	getMeter() + push to readings[]
list_tariffs	TARIFFS constant (direct)
energy_advisor *(prompt)*	summariseUsage() + detectAnomalies()

---

2. Data Layer — src/data/meterStore.ts

Three core interfaces:

Table 2: Core data interfaces and key fields

Interface	Key Fields
HourlyReading	timestamp (ISO8601), kwh, tempC?
Appliance	id, wattHoursPerCycle, dailyCycles, shiftable, activeHours[]
MeterData	meterId, tariffId, readings[], appliances[]

Storage is an **in-memory** Map<string, MeterData> seeded with DEMO_METER (METER-DEMO-001) containing **30 days × 24 hours = 720 synthetic readings** with:

- Cold-weather heating spikes
- Morning/evening peak patterns
- An EV charger running during peak hours

CRUD functions: getMeter(), listMeterIds(), addOrUpdateMeter()

---

3. Tariff Layer — src/data/tariffs.ts

Three built-in tariff plans stored in the TARIFFS record:

Table 3: Built-in tariff plans

ID	Type	Description
flat	Flat rate	Single rate 24/7 (28p/kWh)
tou	Time-of-Use	Peak / off-peak / shoulder windows
dynamic	Dynamic/Agile	Hour-by-hour price variation

getRateForHour(tariff, hour) maps a UTC hour to its applicable rate — this is the primitive used by all cost calculations.

---

4. Analysis Engine — src/analysis/energyAnalysis.ts

Pure functions, no side effects — all take MeterData and return typed result objects:

Table 4: Analysis engine functions and outputs

Function	Output Interface	What it does
summariseUsage()	UsageSummary	30-day rolling totals, avg daily, peak UTC hour
explainBill()	BillExplanation	Root-cause factors (weather, peak usage, top appliances), % vs baseline
recommendSchedules()	ScheduleRecommendation[]	Off-peak shift options for shiftable appliances + annual saving £
detectAnomalies()	Anomaly[]	Spike / sustained-high / overnight-drain detection with severity
buildHourlyProfile()	HourlyProfile[]	24-slot average kWh + cost profile (feeds charts)

---

5. Dashboard — dashboard.html

A **standalone browser HTML file** (no build step) that renders the hourly consumption profile as a chart. It's separate from the MCP server and intended as a visual companion.

Data Flow

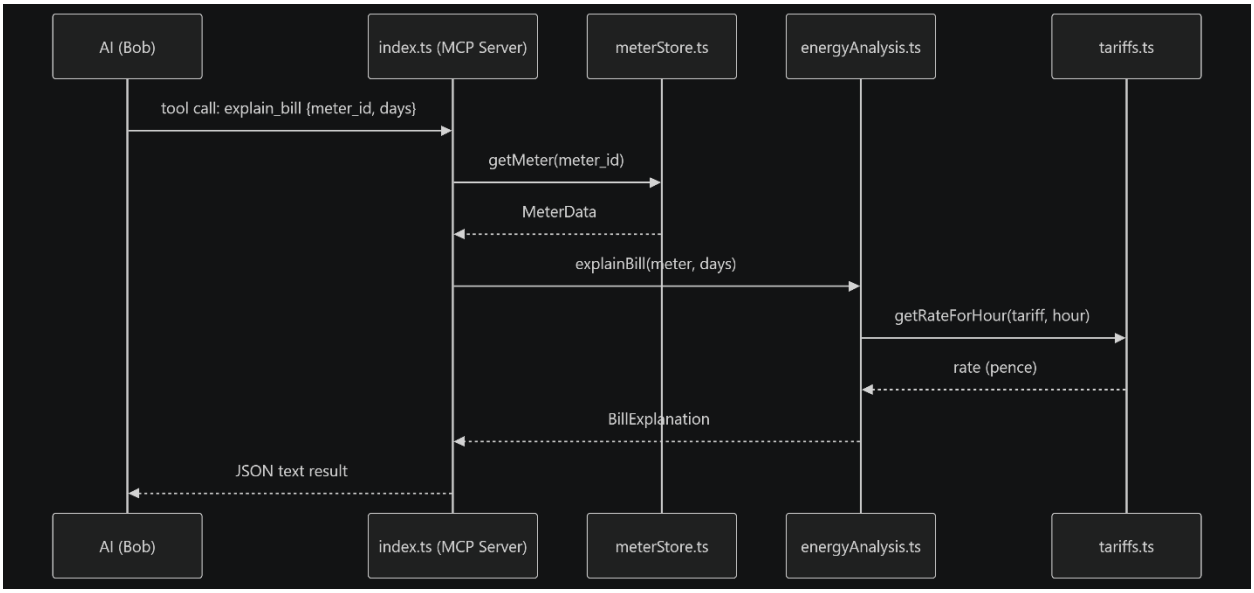


Figure 2: Smart Home Energy Advisor data flow

Key Dependencies

Table 5: Key dependencies

Package	Role
@modelcontextprotocol/server ^2.0	MCP server framework — McpServer, StdioServerTransport, registerTool, registerPrompt
zod ^4.2	Runtime schema validation for all tool inputs
typescript ^5.5	Build-time type safety
@types/node ^22	Node.js type definitions

Technologies Used

Language & Runtime

Table 6: Language and runtime technologies

Technology	Version	Role
TypeScript	^5.5	Primary language — all source files in src/
Node.js	^22 (types)	Runtime environment
ESM (ES Modules)	"type": "module"	Module system (import/export, .mjs)

Core Framework

Table 7: Core framework technologies

Technology	Version	Role
Model Context Protocol (MCP)	@modelcontextprotocol/server ^2.0	Server framework — exposes tools & prompts to AI hosts via stdio transport
@modelcontextprotocol/core	(peer dep)	Core MCP protocol types and transport primitives

---

Validation & Schema

Table 8: Validation and schema technologies

Technology	Version	Role
Zod	^4.2	Runtime schema validation for all tool input parameters

---

Build Tooling

Table 9: Build tooling technologies

Technology	Version	Role
TypeScript Compiler (tsc**)**	^5.5	Compiles src/ → build/ (CommonJS output)
npm	—	Package management (package-lock.json)

Transport / Communication

Table 10: Transport and communication

Technology	Role
stdio (stdin/stdout)	MCP server transport — the AI host communicates with the server over standard I/O using JSON-RPC

Frontend

Table 11: Frontend technologies

Technology	Role
Vanilla HTML/CSS/JS	dashboard.html — standalone browser chart dashboard, no framework, no build step

Architecture Patterns

Table 12: Architecture patterns

Pattern	Where
Layered architecture	Entry point → Analysis engine → Data layer
Pure functions	All analysis logic in energyAnalysis.ts — no side effects
In-memory store	meterStore.ts uses a Map<string, MeterData>
Schema-first tool design	Every MCP tool defines its contract via Zod before any logic runs

In short: it's a **TypeScript + Node.js MCP server** with **Zod** for validation, built and served as an **ESM module over stdio**, with no database and no web framework — just clean in-memory data structures and pure analysis functions.

All Files Created in the Smart Home Energy Advisor Task

Source Code Files

Table 13: Source code files

File	Purpose
src/index.ts	MCP server entry point — registers all 8 tools and 1 prompt
src/data/meterStore.ts	In-memory data store — MeterData, HourlyReading, Appliance interfaces + 30-day demo data
src/data/tariffs.ts	Tariff definitions — flat, time-of-use, dynamic agile + getRateForHour()
src/analysis/energyAnalysis.ts	Pure analysis engine — summariseUsage, explainBill, recommendSchedules, detectAnomalies, buildHourlyProfile

---

Configuration Files

Table 14: Configuration files

File	Purpose
package.json	Project metadata, dependencies (@modelcontextprotocol/server, zod), build scripts
package-lock.json	Locked dependency tree for reproducible installs
tsconfig.json	TypeScript compiler configuration — compiles src/ → build/

Build Output

Table 15: Build output

File	Purpose

File	Purpose
index.mjs	Pre-built ESM entry point — runnable directly without the build step

---

Frontend

Table 16: Frontend files

File	Purpose
dashboard.html	Standalone browser chart dashboard — visualises hourly consumption profile, no build step required

---

Documentation

Table 17: Documentation files

File	Purpose
README.md	Setup instructions, tool reference table, architecture overview, MCP registration config

---

Documents Created in This Session

Table 18: Documents created in this session

File	Purpose
smart-home-energy-advisor.pptx	13-slide PowerPoint presentation covering architecture, tools, agentic AI, novelty, future scope
smart-home-energy-advisor-problem-solution.docx	11-section Word document — full problem statement and solution document

---

Directory Structure at a Glance

smart-home-energy-advisor/

```
│
├── src/                          ← TypeScript source
│   ├── index.ts                  ← MCP server entry point
│   ├── data/
│   │   ├── meterStore.ts         ← Data store + demo data
│   │   └── tariffs.ts            ← Tariff plans
│   └── analysis/
│       └── energyAnalysis.ts     ← Pure analysis engine
│
├── package.json                  ← Project config
├── package-lock.json             ← Locked dependencies
├── tsconfig.json                 ← TypeScript config
├── index.mjs                     ← Pre-built ESM entry
├── dashboard.html                ← Browser chart UI
├── README.md                     ← Documentation
│
├── smart-home-energy-advisor.pptx       ← Presentation (this session)
└── smart-home-energy-advisor-problem-solution.docx  ← Problem/Solution doc (this session)
```

File Count Summary

Table 19: File count summary

Category	Count

Category	Count
TypeScript source files	4
Configuration files	3
Build output	1
Frontend	1
Documentation	1
Office documents (this session)	2
Total	12

IBM Bob Screenshots:

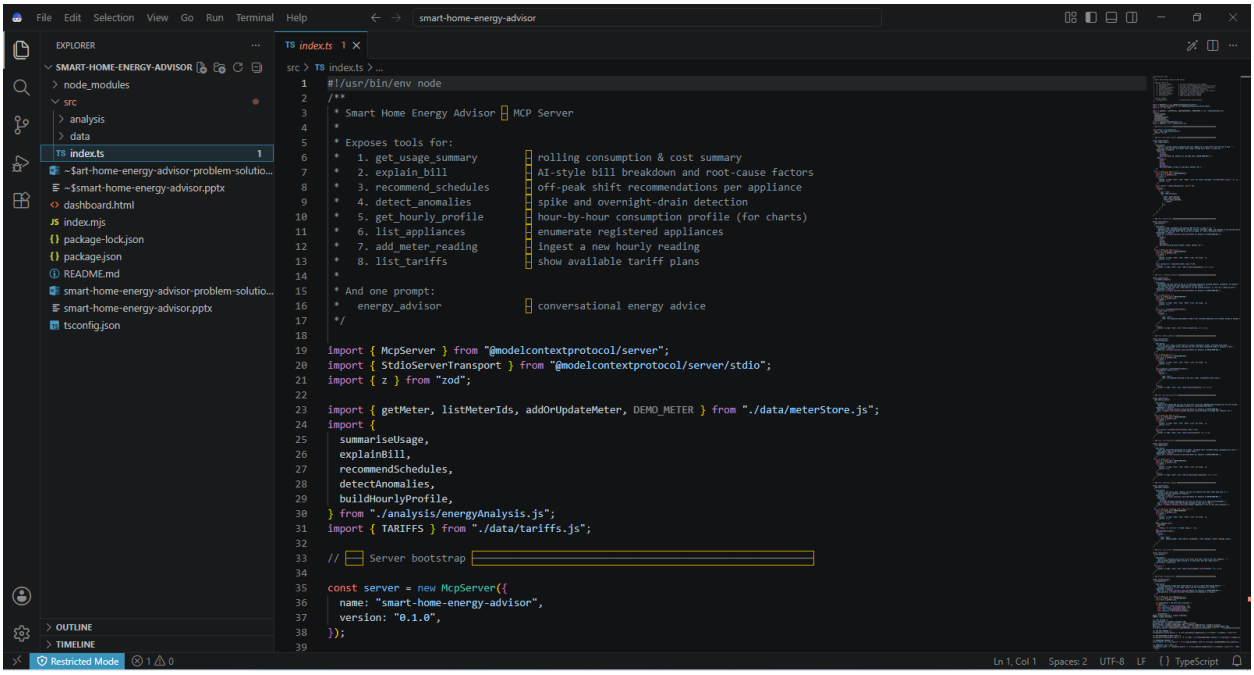


Figure 3: IBM Bob project/code screenshot 1

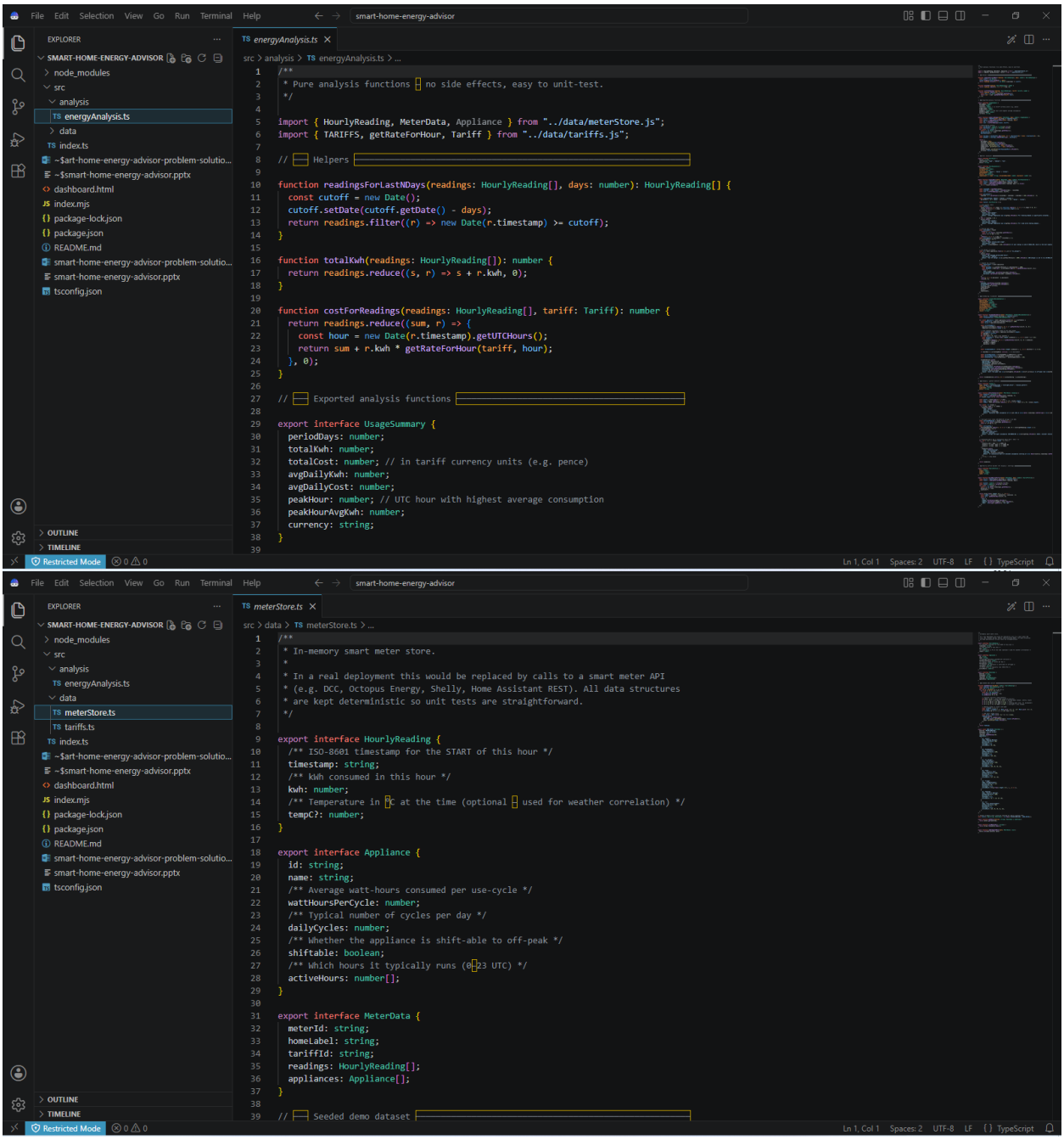


Figure 4: IBM Bob project/code screenshot 2

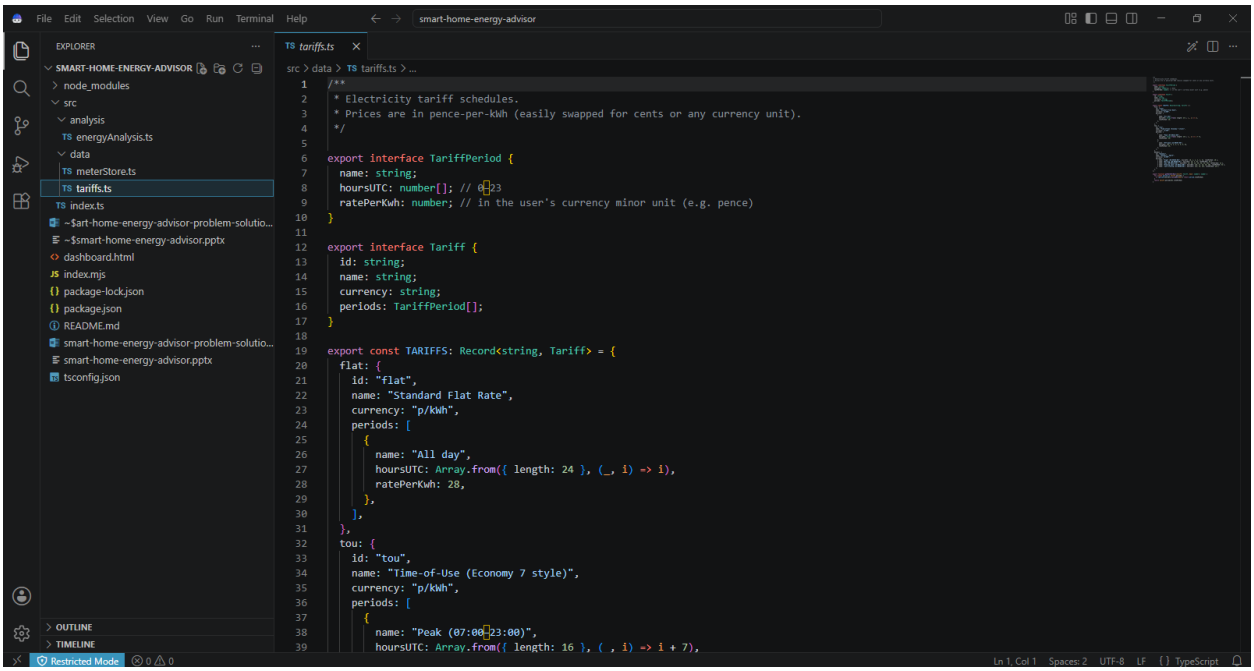


Figure 5: IBM Bob project/code screenshot 3

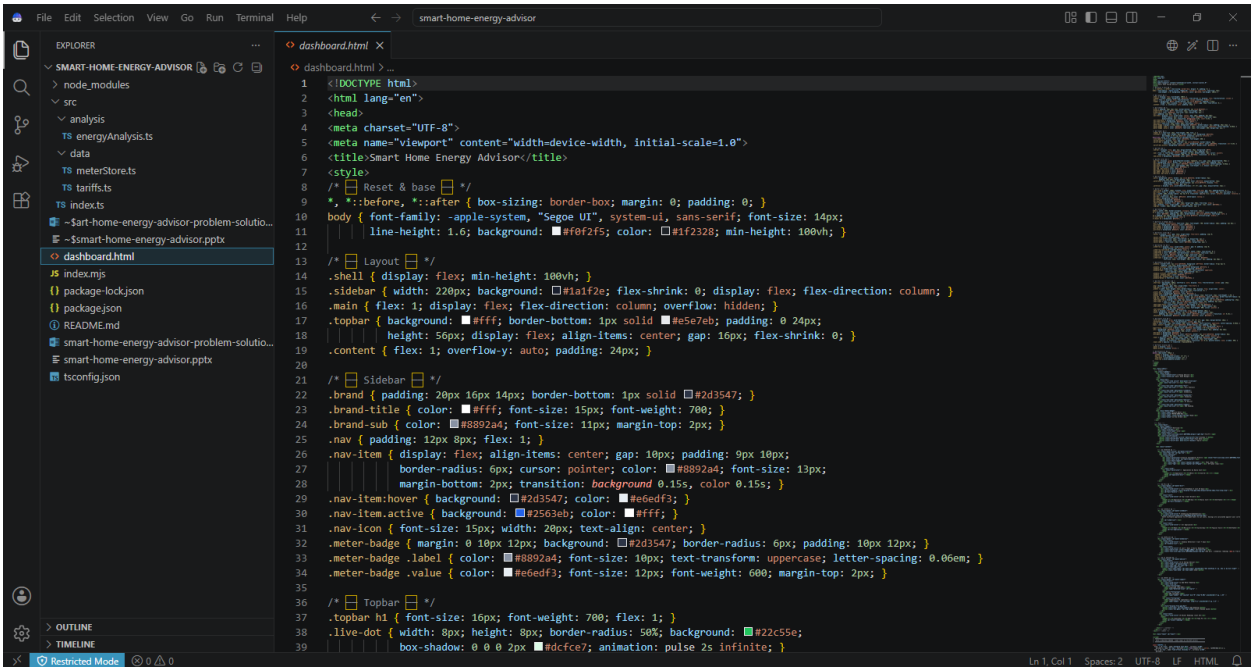
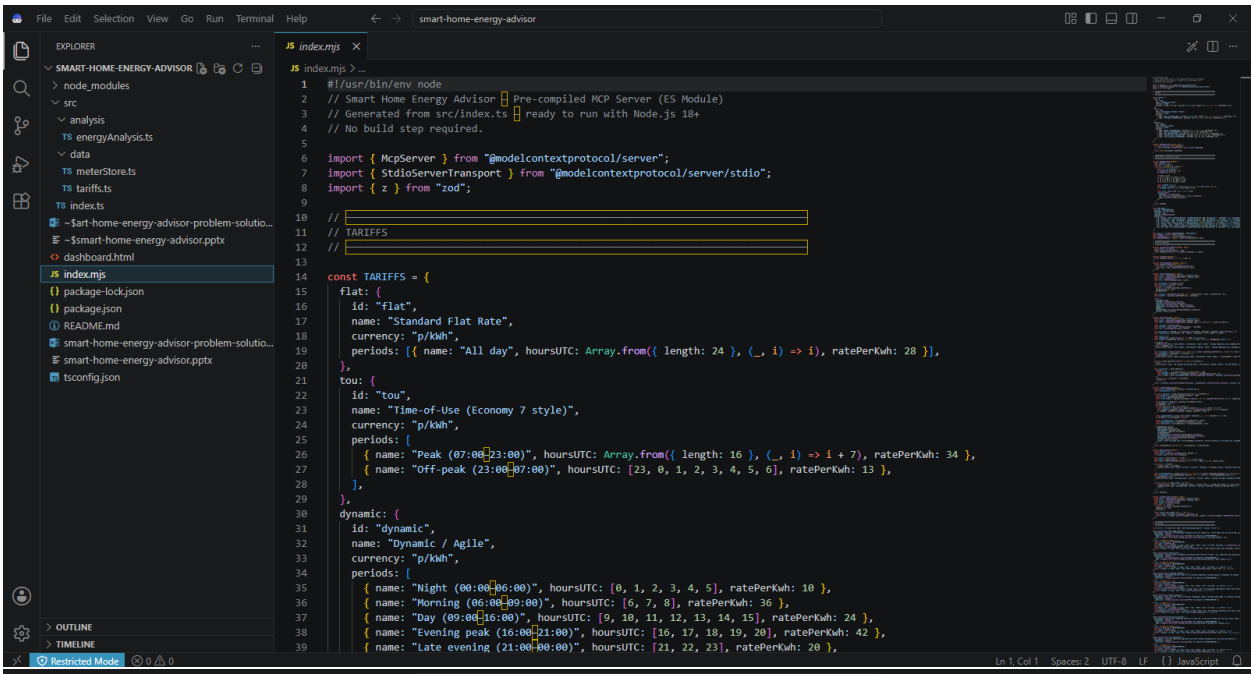
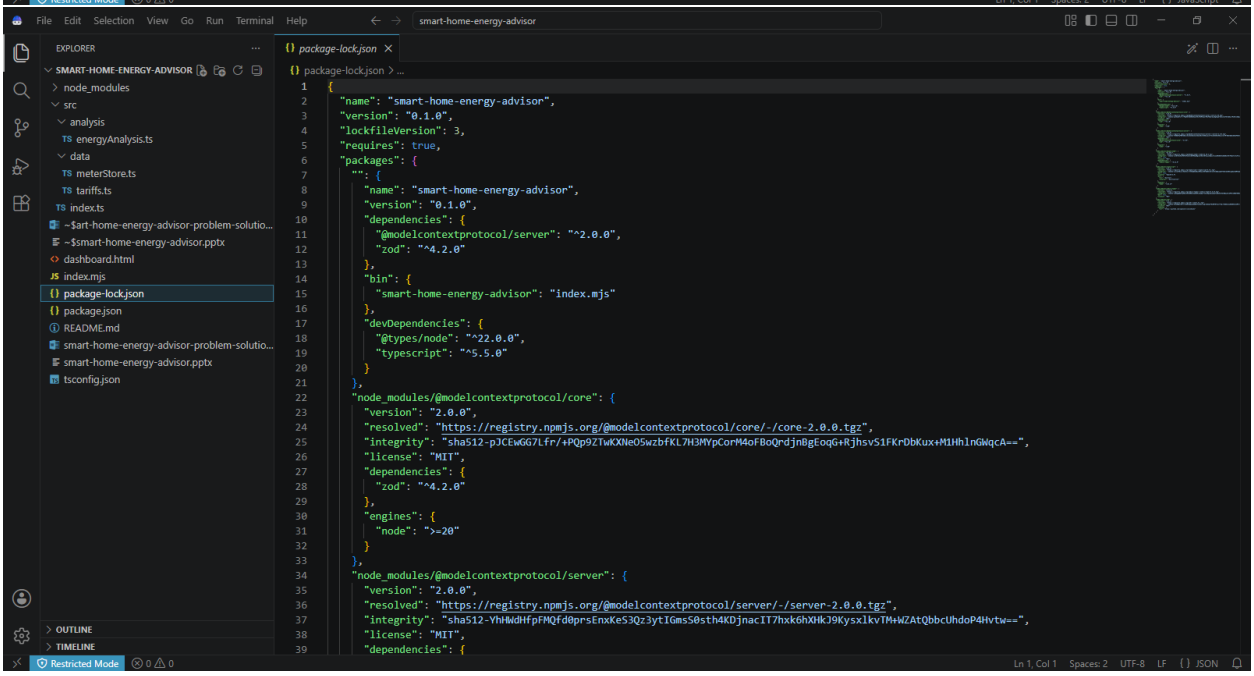


Figure 6: IBM Bob project/code screenshot 4


File Edit Selection View Go Run Terminal Help
smart-home-energy-advisor
EXPLORER
SMART-HOME-ENERGY-ADVISOR
node_modules
src
analysis
energyAnalysis.ts
data
meterStore.ts
tariffs.ts
index.ts
~$art-home-energy-advisor-problem-solutio...
~$smart-home-energy-advisor.pptx
dashboard.html
index.mjs
package-lock.json
package.json
README.md
smart-home-energy-advisor-problem-solutio...
smart-home-energy-advisor.pptx
tsconfig.json
OUTLINE
TIMELINE
index.mjs
#!/usr/bin/env node
// Smart Home Energy Advisor — Pre-compiled MCP Server (ES Module)
// Generated from src/index.ts — ready to run with Node.js 18+
// No build step required.
import { McpServer } from "@modelcontextprotocol/server";
import { StdioServerTransport } from "@modelcontextprotocol/server/stdio";
import { z } from "zod";
// TARIFFS
const TARIFFS = {
flat: {
id: "flat",
name: "Standard Flat Rate",
currency: "p/kWh",
periods: [{ name: "All day", hoursUTC: Array.from({ length: 24 }, (_, i) => i), ratePerKwh: 28 }],
},
tou: {
id: "tou",
name: "Time-of-Use (Economy 7 style)",
currency: "p/kWh",
periods: [
{ name: "Peak (07:00–23:00)", hoursUTC: Array.from({ length: 16 }, (_, i) => i + 7), ratePerKwh: 34 },
{ name: "Off-peak (23:00–07:00)", hoursUTC: [23, 0, 1, 2, 3, 4, 5, 6], ratePerKwh: 13 },
],
},
dynamic: {
id: "dynamic",
name: "Dynamic / Agile",
currency: "p/kWh",
periods: [
{ name: "Night (00:00–06:00)", hoursUTC: [0, 1, 2, 3, 4, 5], ratePerKwh: 10 },
{ name: "Morning (06:00–09:00)", hoursUTC: [6, 7, 8], ratePerKwh: 36 },
{ name: "Day (09:00–16:00)", hoursUTC: [9, 10, 11, 12, 13, 14, 15], ratePerKwh: 24 },
{ name: "Evening peak (16:00–21:00)", hoursUTC: [16, 17, 18, 19, 20], ratePerKwh: 42 },
{ name: "Late evening (21:00–00:00)", hoursUTC: [21, 22, 23], ratePerKwh: 20 },
Restricted Mode
Ln 1, Col 1 Spaces: 2 UTF-8 LF { } JavaScript


File Edit Selection View Go Run Terminal Help
smart-home-energy-advisor
EXPLORER
SMART-HOME-ENERGY-ADVISOR
node_modules
src
analysis
energyAnalysis.ts
data
meterStore.ts
tariffs.ts
index.ts
~$art-home-energy-advisor-problem-solutio...
~$smart-home-energy-advisor.pptx
dashboard.html
index.mjs
package-lock.json
package.json
README.md
smart-home-energy-advisor-problem-solutio...
smart-home-energy-advisor.pptx
tsconfig.json
OUTLINE
TIMELINE
package-lock.json
{
"name": "smart-home-energy-advisor",
"version": "0.1.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
"": {
"name": "smart-home-energy-advisor",
"version": "0.1.0",
"dependencies": {
"@modelcontextprotocol/server": "^2.0.0",
"zod": "^4.2.0"
},
"bin": {
"smart-home-energy-advisor": "index.mjs"
},
"devDependencies": {
"@types/node": "^22.0.0",
"typescript": "^5.5.0"
}
},
"node_modules/@modelcontextprotocol/core": {
"version": "2.0.0",
"resolved": "https://registry.npmjs.org/@modelcontextprotocol/core/-/core-2.0.0.tgz",
"integrity": "sha512-pJCEwGG7Lfr/+PQp9ZTwKXNeO5wzbfKL7H3MYpCorM4oFBoQrdjnBgEoqG+RjhsvS1FKrDbKux+M1HhlnGWqcA==",
"license": "MIT",
"dependencies": {
"zod": "^4.2.0"
},
"engines": {
"node": ">=20"
}
},
"node_modules/@modelcontextprotocol/server": {
"version": "2.0.0",
"resolved": "https://registry.npmjs.org/@modelcontextprotocol/server/-/server-2.0.0.tgz",
"integrity": "sha512-YhHWdHfpFMQfd0prsEnxKeS3Qz3ytIGmsS0sth4KDjnacIT7hxk6hXHkJ9KysxlkvTM+WZAtQbbcUhdoP4Hvtw==",
"license": "MIT",
"dependencies": {
Restricted Mode
Ln 1, Col 1 Spaces: 2 UTF-8 LF { } JSON

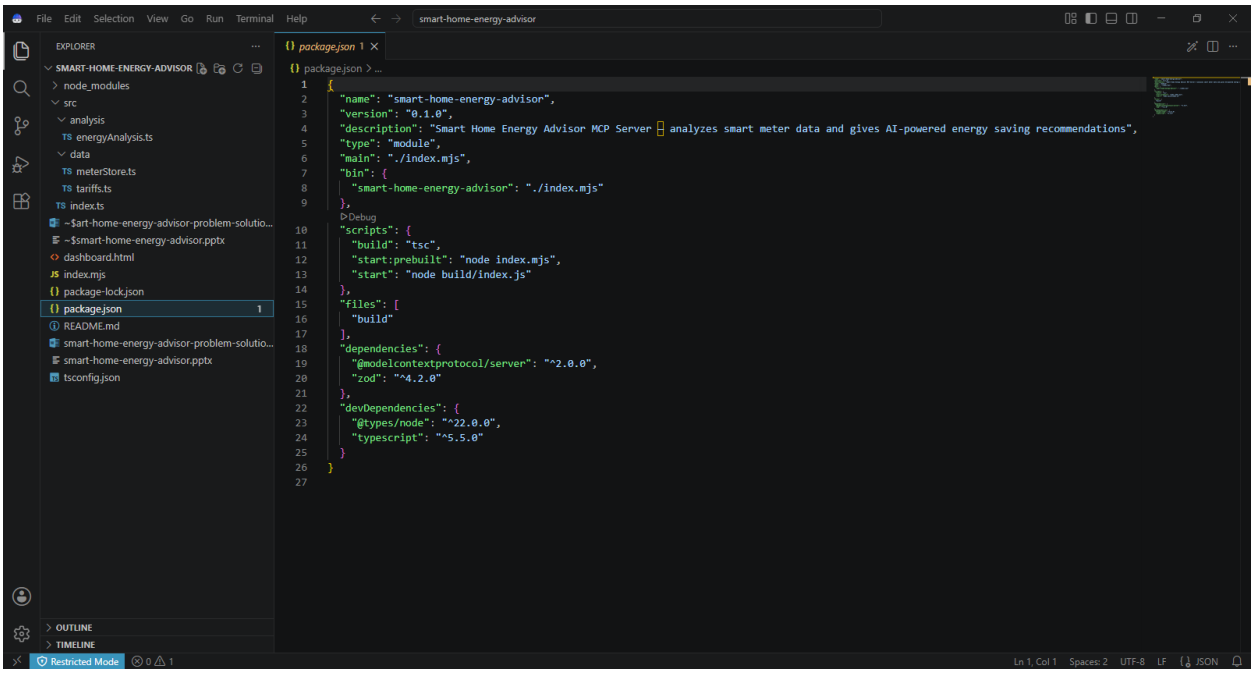


Figure 7: IBM Bob project/code screenshot 5

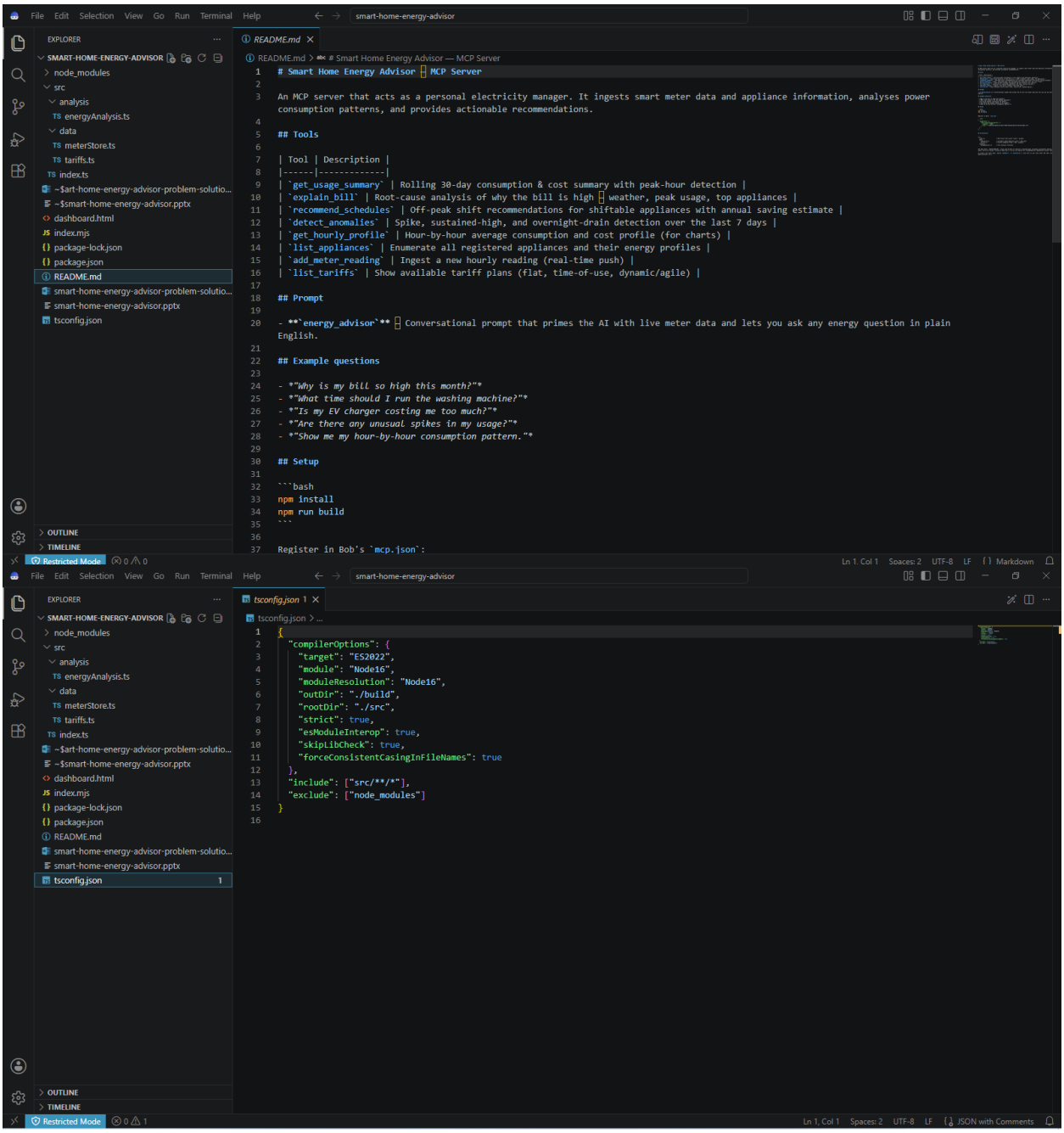


Figure 8: IBM Bob project/code screenshot 6

IBM Bob — Detailed Architecture

What Is IBM Bob?

IBM Bob is an AI coding assistant available in two deployment forms:

- **Bob IDE — a chat panel embedded in your code editor**
- **Bob Shell — a terminal-first CLI (bob) for command-line workflows**

Both share the same core AI reasoning engine, context model, and extensibility system, but differ in their available modes and tool surface.

High-Level Architecture

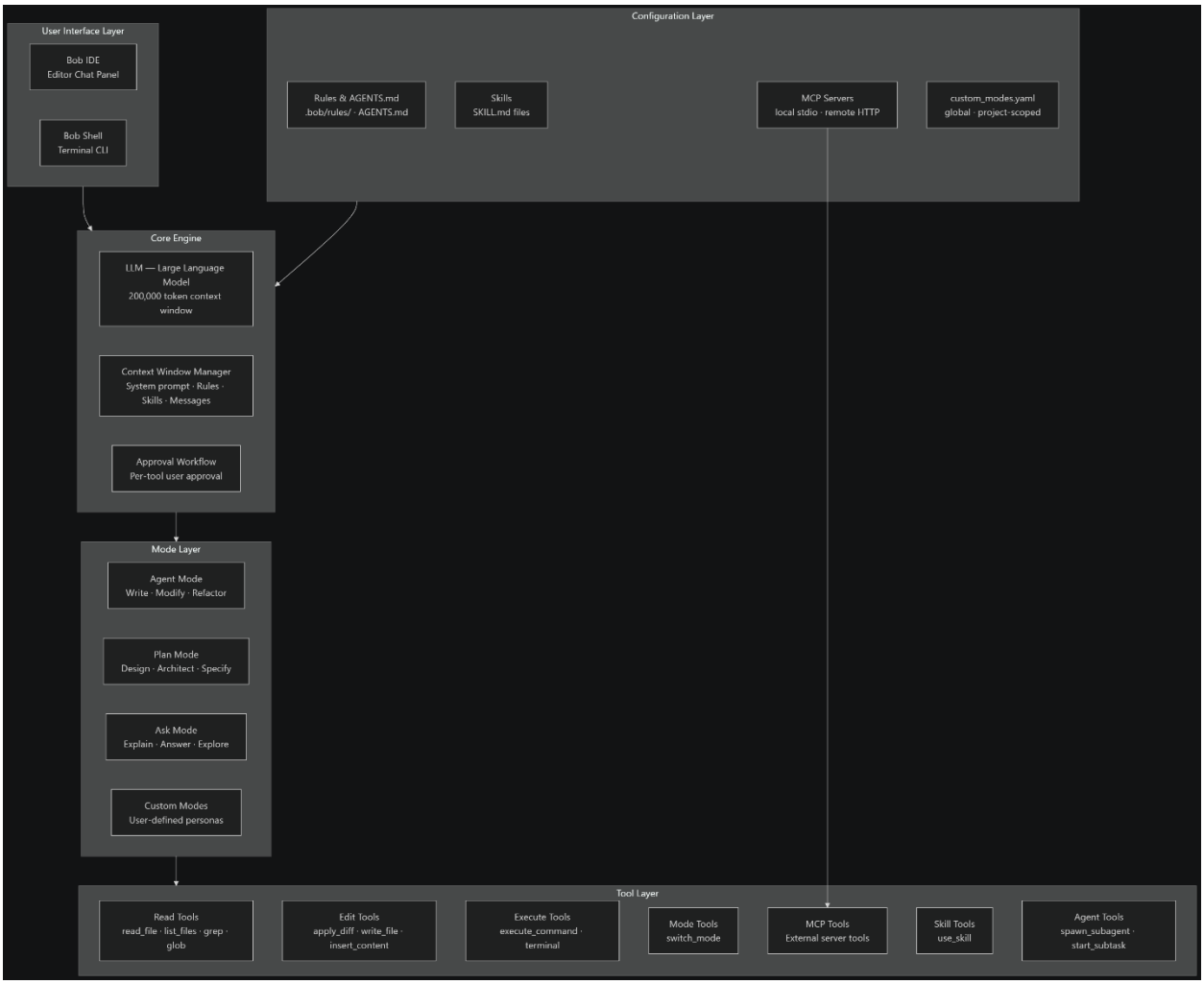


Figure 9: IBM Bob project/code screenshot 7

1. User Interface Layer

Table 20: IBM Bob interface variants

Variant	Description
Bob IDE	Chat panel inside a supported code editor. Full agentic mode with file explorer, diff viewer, approval UI, token usage indicator
Bob Shell	Terminal CLI (bob). Same AI, optimised for shell-based workflows — scripting, automation, CI pipelines

---

2. Core Engine

LLM & Context Window

The model powering Bob provides a 200,000-token context window. Every conversation starts fresh with that full capacity. Bob automatically summarises older content when the window fills.

The context window is divided into 6 categories, each consuming tokens before you even type:

Table 21: Bob context-window categories

Category	What It Contains	Dynamic?
System Prompt	Bob's core persona and base instructions	Fixed per session
Tool Definitions	Schemas for all built-in tools	Fixed per session
MCP Tools	Descriptions of tools from connected MCP servers	Grows when servers added
Rules	AGENTS.md and .bob/rules/*.md instructions	Fixed when task opens
Skills	Loaded skill instructions (SKILL.md)	Can grow mid-thread
Messages	User prompts, Bob replies, tool calls, @ mentions	Grows every turn

On a baseline empty session, overhead is already ~8.5k tokens before the first message (tool definitions ~5.1k, system prompt ~1.5k, rules ~830, skills ~454).

Approval Workflow

Before any tool executes, Bob surfaces an approval prompt — one at a time. Users can:

- **Approve individually**
- **Edit the proposed terminal command inline before approving**
- **Configure auto-approval per tool group (read / edit / execute) globally or per task**

---

3. Mode Layer

Modes are purpose-built personas that determine Bob's behaviour, tool access, and role definition.

Built-in Modes (Bob IDE)

Table 22: Bob IDE built-in modes

Mode	Purpose	Key Tool Access
Agent	Write, modify, refactor code	read + edit + execute + mcp + skill + subagent
Plan	Design, architect, strategise	read + skill (no file writes)
Ask	Explain, answer, explore	read only

Built-in Modes (Bob Shell)

Table 23: Bob Shell built-in modes

Mode	Purpose
Code	Generate, modify, refactor from terminal
Ask	Answer codebase questions

Mode	Purpose
Plan	Design before running
Advanced	Extended capabilities including MCP

Custom Modes

Defined in ~/.bob/settings/custom_modes.yaml (global) or .bob/custom_modes.yaml (project). Each custom mode specifies:

```
customModes:
  - slug: docs-writer
    name: 📝 Documentation Writer
    roleDefinition: You are a technical writer...
    whenToUse: Use for writing and editing docs.
    customInstructions: Focus on clarity...
    groups:
      - read
      - [edit, fileRegex: ".*\\.(md|mdx)$"]  # restrict edits to .md only
      - skill
```

Project-scoped modes override global modes, which override defaults.

---

4. Tool Layer

Tools are how Bob interacts with the system. Bob selects them autonomously — users never call tools directly, only approve them.

Tool Groups & Permissions

Table 24: Tool groups and permissions

Group	Tools Included	Permission

Group	Tools Included	Permission
read	read_file, list_files, grep, glob, GetSymbolsOverview, FindSymbol	Read-only filesystem
edit	write_file, apply_diff, search_and_replace, insert_content	File modification (can be fileRegex-restricted)
execute	execute_command	Terminal command execution
mcp	All tools from connected MCP servers	External services
skill	use_skill	Load skill instruction files
mode	switch_mode	Switch between modes
todo	update_todo_list	Task tracking
subtask	start_subtask	Create child tasks
subagent	spawn_subagent	Spawn independent sub-agents
workflow	start_workflow	Launch pre-defined workflows

5. MCP (Model Context Protocol) Layer

MCP is the extensibility backbone of Bob. It uses a client-server architecture:

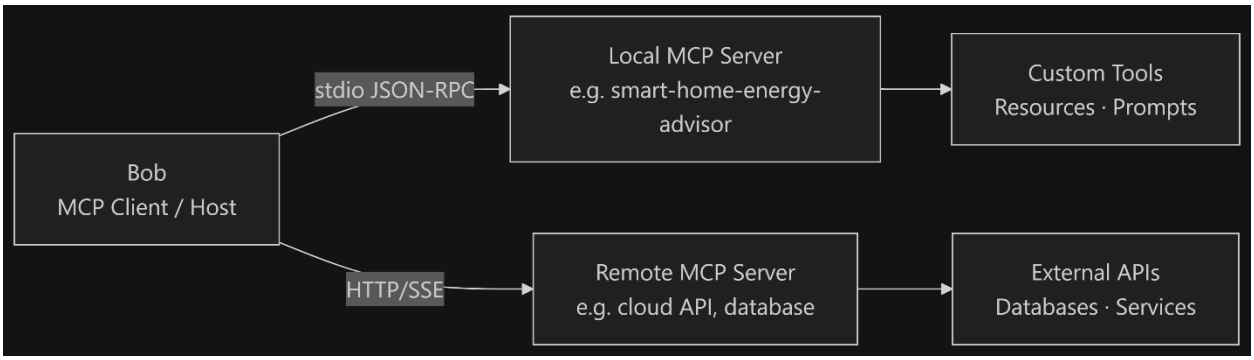


Figure 10: IBM Bob project/code screenshot 8

- **Bob acts as the MCP host/client — it connects to MCP servers**
- **MCP servers can be local (stdio transport) or remote (HTTP/SSE)**
- **Individual tools within a server can be enabled or disabled to manage context window usage**
- **Configured in mcp.json at global or project scope**

---

6. Configuration Layer

All of Bob's behaviour is shaped by four configuration mechanisms:

AGENTS.md & Rules

Loaded hierarchically — global → project root → subdirectory:

~/.bob/AGENTS.md ← Global, applies to all projects

AGENTS.md ← Project root

.bob/rules/custom_rules.md ← Additional project rules

src/components/AGENTS.md ← Component-specific rules

Rules apply to every conversation regardless of mode. They are the correct mechanism for cross-cutting behaviours (coding standards, test commands, style guides).

Skills

Reusable instruction packs stored as SKILL.md files. Activated mid-conversation via use_skill. Skills fill the Skills category in the context window and are loaded on demand — not all at once.

MCP Servers

Configured in mcp.json. Each server entry specifies transport (stdio command or HTTP URL), environment variables, and optional tool enable/disable lists.

Custom Modes

Stored in custom_modes.yaml. Override default modes by matching their slug. Scope is global or project-level.

Full Architecture at a Glance

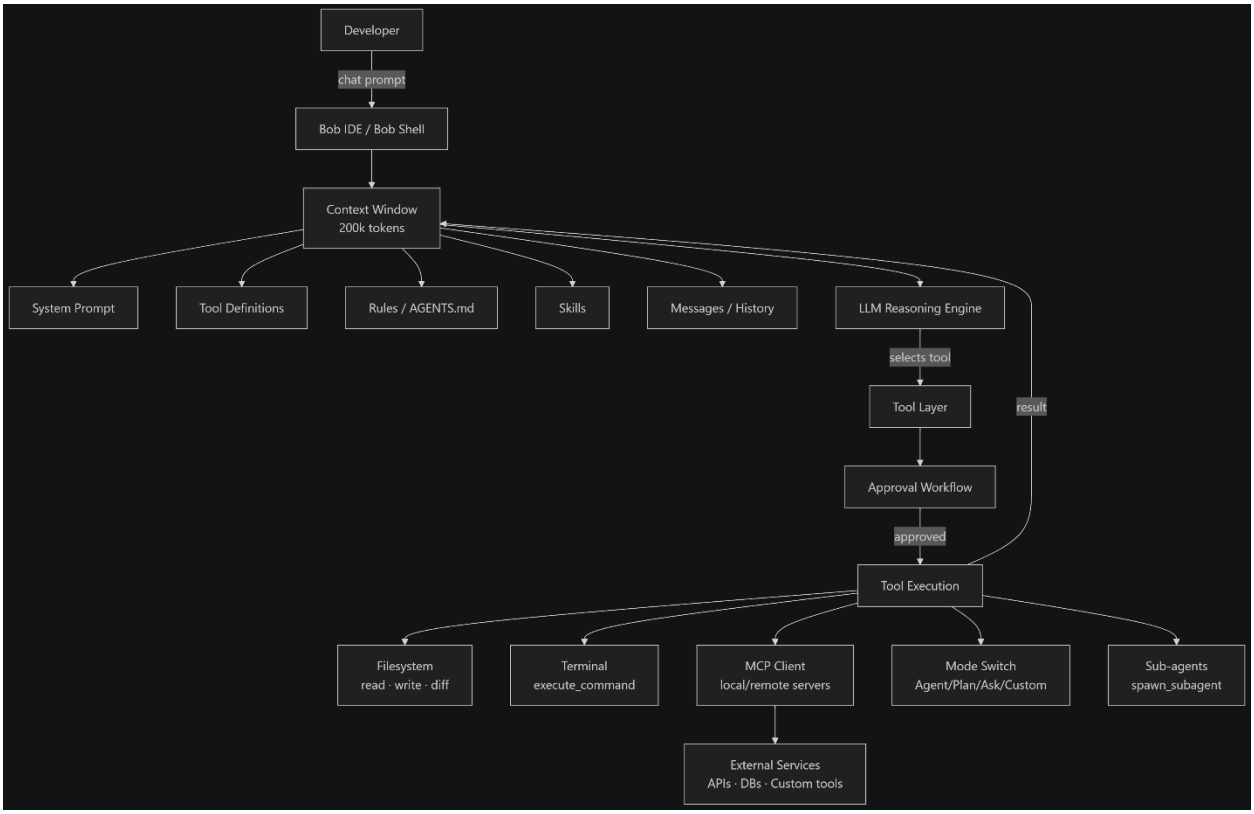


Figure 11: IBM Bob project/code screenshot 9

Summary Table

Table 25: IBM Bob architectural components

Architectural Component	Role
LLM	Reasoning, tool selection, code generation
Context Window (200k tokens)	Working memory — holds instructions, rules, history, tool output
Modes	Persona + tool permission sets (Agent / Plan / Ask / Custom)
Tool Groups	Granular capability grants (read / edit / execute / mcp / skill…)
Approval Workflow	Human-in-the-loop control before every tool execution
MCP Protocol	Extensibility layer — connects Bob to any external service
AGENTS.md / Rules	Persistent project-specific instructions loaded at session start
Skills	On-demand instruction packs for specialised tasks
Custom Modes	Team-defined personas with scoped tool access and role definitions
Bob Shell	Same engine surfaced as a terminal CLI for automation and CI

Project Screenshots

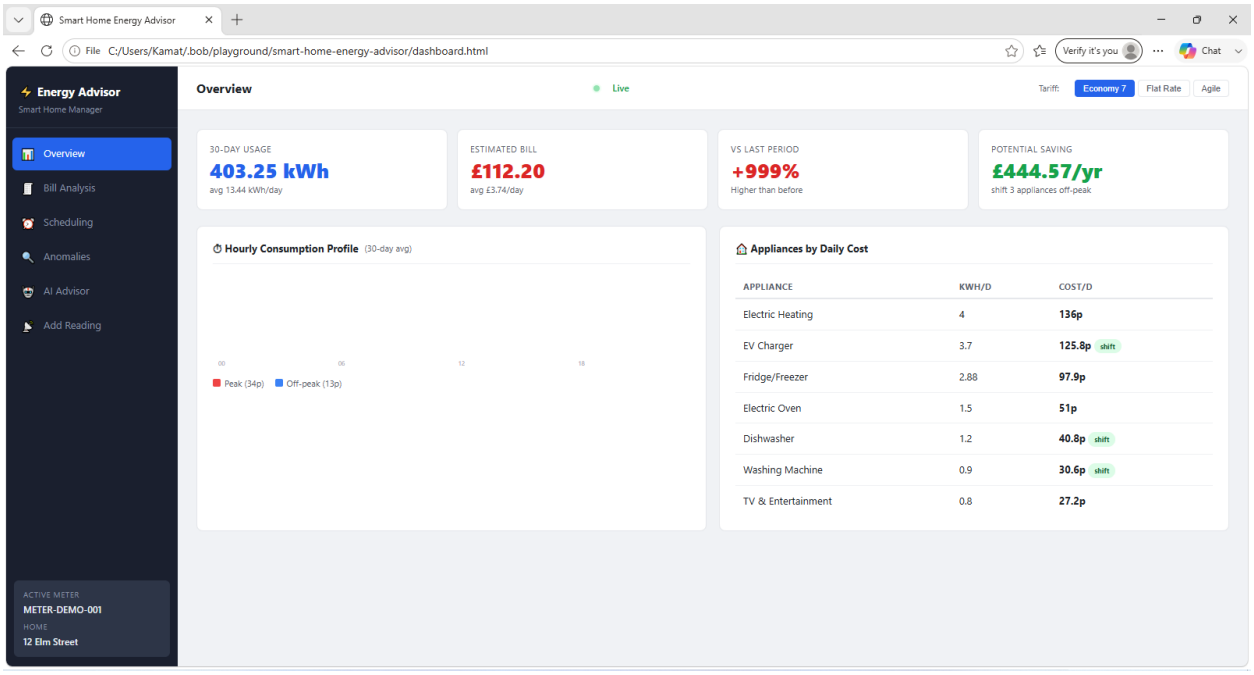


Figure 12: IBM Bob project/code screenshot 10

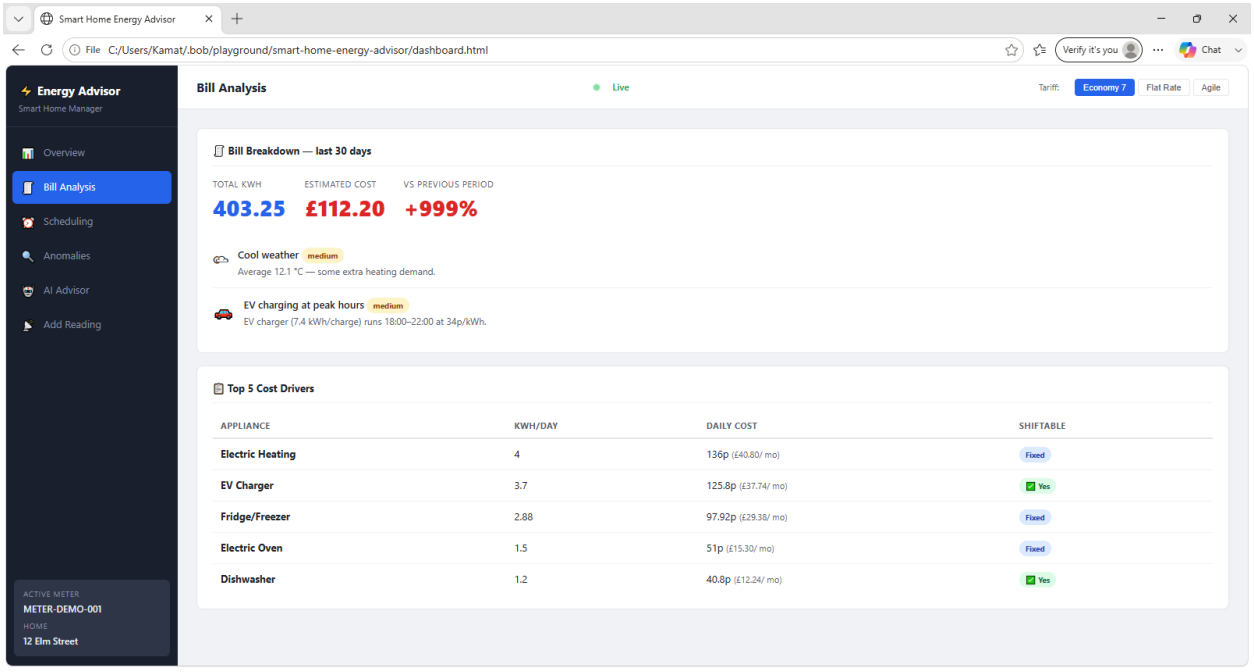


Figure 13: IBM Bob high-level architecture

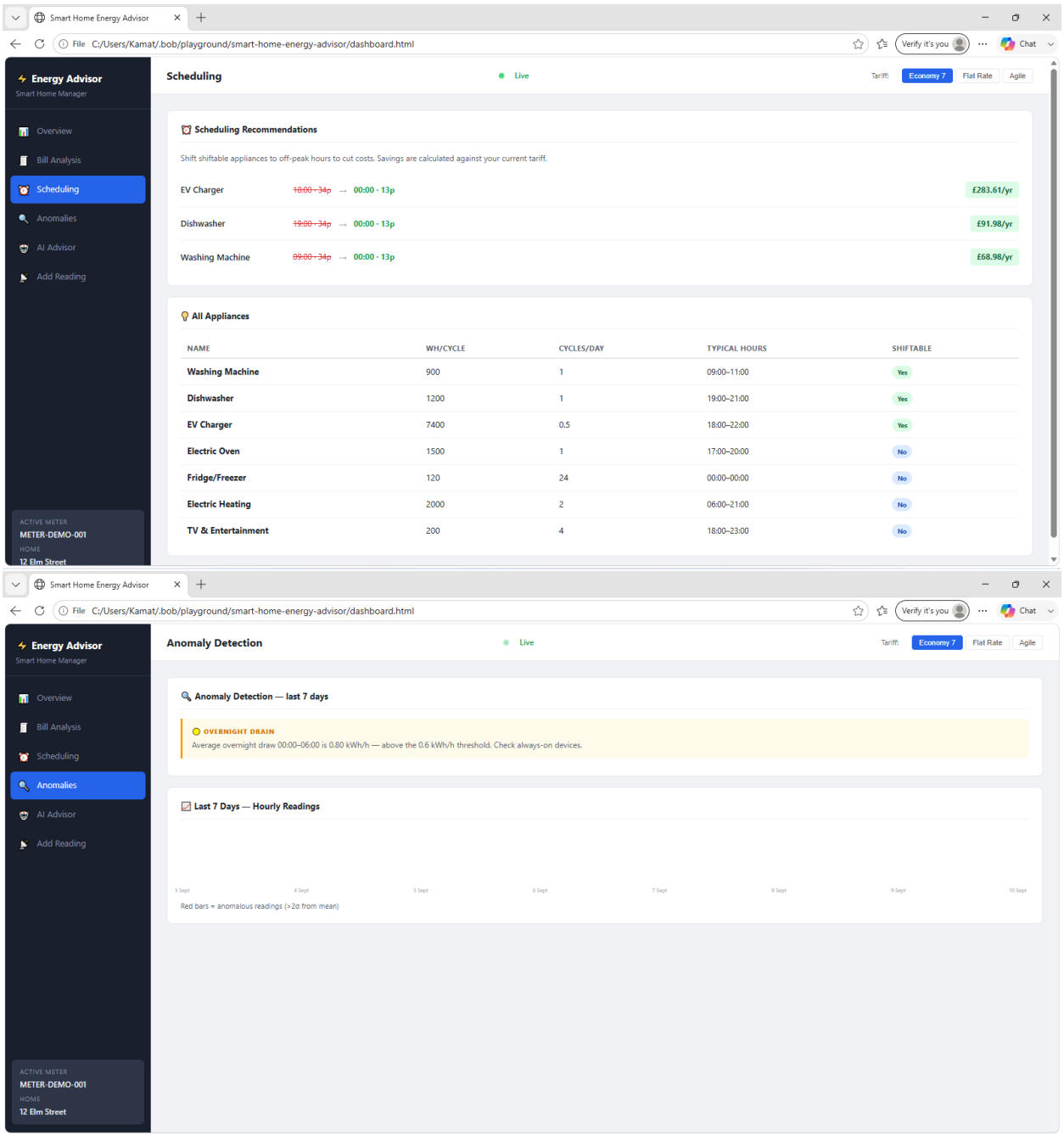
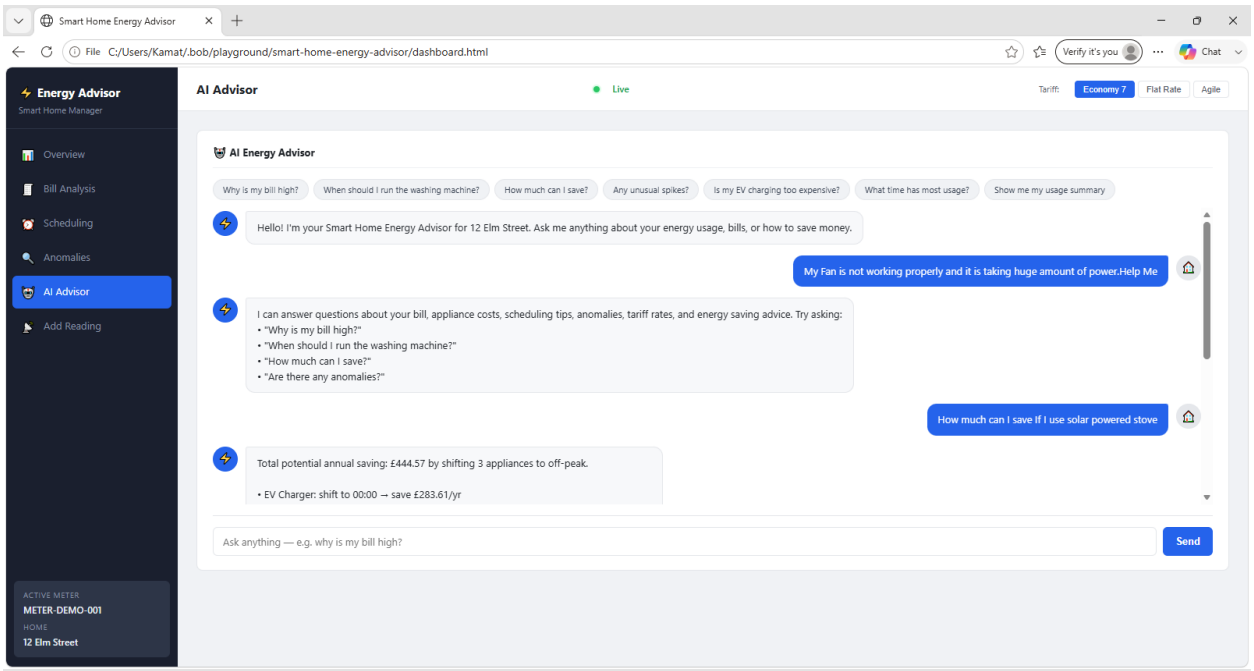
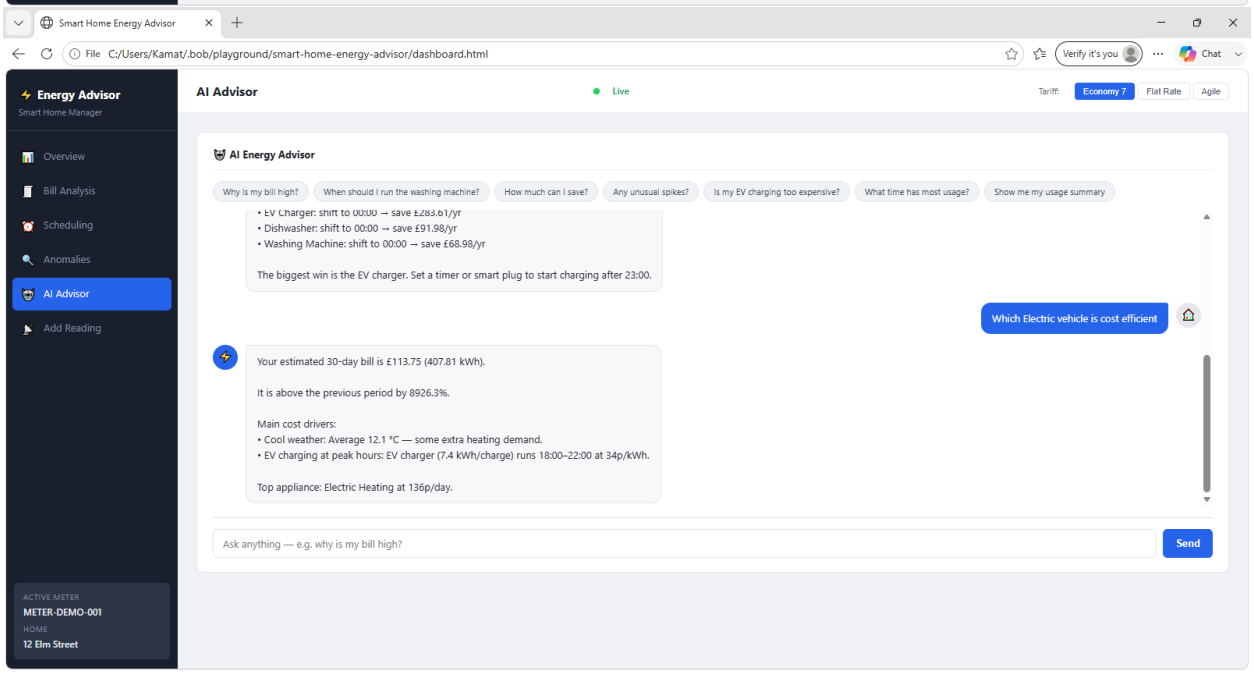


Figure 14: MCP client-server architecture


Smart Home Energy Advisor
File C:/Users/Kamat/.bob/playground/smart-home-energy-advisor/dashboard.html
Verify it's you
Chat
Energy Advisor
Smart Home Manager
Overview
Bill Analysis
Scheduling
Anomalies
AI Advisor
Add Reading
ACTIVE METER
METER-DEMO-001
HOME
12 Elm Street
AI Advisor
Live
Tariff:
Economy 7
Flat Rate
Agile
AI Energy Advisor
Why is my bill high?
When should I run the washing machine?
How much can I save?
Any unusual spikes?
Is my EV charging too expensive?
What time has most usage?
Show me my usage summary
Hello! I'm your Smart Home Energy Advisor for 12 Elm Street. Ask me anything about your energy usage, bills, or how to save money.
My Fan is not working properly and it is taking huge amount of power.Help Me
I can answer questions about your bill, appliance costs, scheduling tips, anomalies, tariff rates, and energy saving advice. Try asking:
• "Why is my bill high?"
• "When should I run the washing machine?"
• "How much can I save?"
• "Are there any anomalies?"
How much can I save If I use solar powered stove
Total potential annual saving: £444.57 by shifting 3 appliances to off-peak.
• EV Charger: shift to 00:00 → save £283.61/yr
Ask anything — e.g. why is my bill high?
Send


Smart Home Energy Advisor
File C:/Users/Kamat/.bob/playground/smart-home-energy-advisor/dashboard.html
Verify it's you
Chat
Energy Advisor
Smart Home Manager
Overview
Bill Analysis
Scheduling
Anomalies
AI Advisor
Add Reading
ACTIVE METER
METER-DEMO-001
HOME
12 Elm Street
AI Advisor
Live
Tariff:
Economy 7
Flat Rate
Agile
AI Energy Advisor
Why is my bill high?
When should I run the washing machine?
How much can I save?
Any unusual spikes?
Is my EV charging too expensive?
What time has most usage?
Show me my usage summary
• EV Charger: shift to 00:00 → save £283.61/yr
• Dishwasher: shift to 00:00 → save £91.98/yr
• Washing Machine: shift to 00:00 → save £68.98/yr
The biggest win is the EV charger. Set a timer or smart plug to start charging after 23:00.
Which Electric vehicle is cost efficient
Your estimated 30-day bill is £113.75 (407.81 kWh).
It is above the previous period by 8926.3%.
Main cost drivers:
• Cool weather: Average 12.1 °C — some extra heating demand.
• EV charging at peak hours: EV charger (7.4 kWh/charge) runs 18:00–22:00 at 34p/kWh.
Top appliance: Electric Heating at 136p/day.
Ask anything — e.g. why is my bill high?
Send

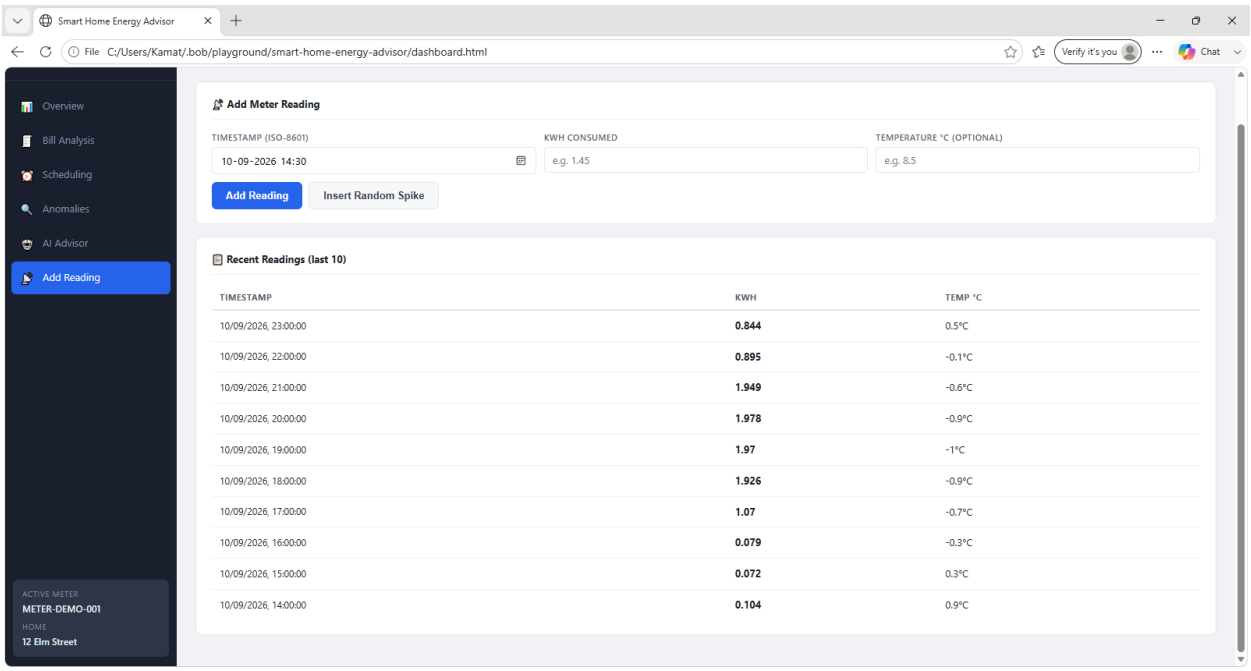


Figure 15: IBM Bob full architecture overview

Role of Agentic AI in Smart Home Energy Advisor

What Makes It "Agentic"?

This project is not just a passive API — it is designed as an **MCP server**, which means an AI agent (like Bob) can autonomously **discover, select, and chain tools** to answer complex energy questions without being explicitly told which tool to call.

---

1. Tool Selection & Reasoning

The AI agent decides **which tool(s) to invoke** based on the user's natural language question:

Table 26: Agent tool selection examples

User Question	Agent Selects

User Question	Agent Selects
"Why is my bill so high?"	explain_bill
"When should I run the washing machine?"	recommend_schedules
"Is something draining power at night?"	detect_anomalies
"Show me my consumption pattern"	get_hourly_profile
"What appliances do I have?"	list_appliances

The agent interprets **intent** → maps it to the right tool → executes it — all autonomously.

---

2. Multi-Step Tool Chaining

For complex queries, the agent can **chain multiple tools** in sequence:

Figure 16: Smart Home Energy Advisor project screenshot 1

No single tool does all of this — the agent **composes** them together into a coherent answer.

---

3. The energy_advisor Prompt — Agentic Priming

The energy_advisor prompt is specifically designed for agentic behavior:

- It **pre-loads live meter context** (current usage summary + any active anomalies) into the AI's system prompt
- It primes the AI to act as a *domain expert*, not just a tool caller
- The agent can then handle **open-ended follow-up questions** using that live context without re-calling tools every turn

---

4. Real-Time Data Ingestion via add_meter_reading

The agent can **act on incoming data** in real time:

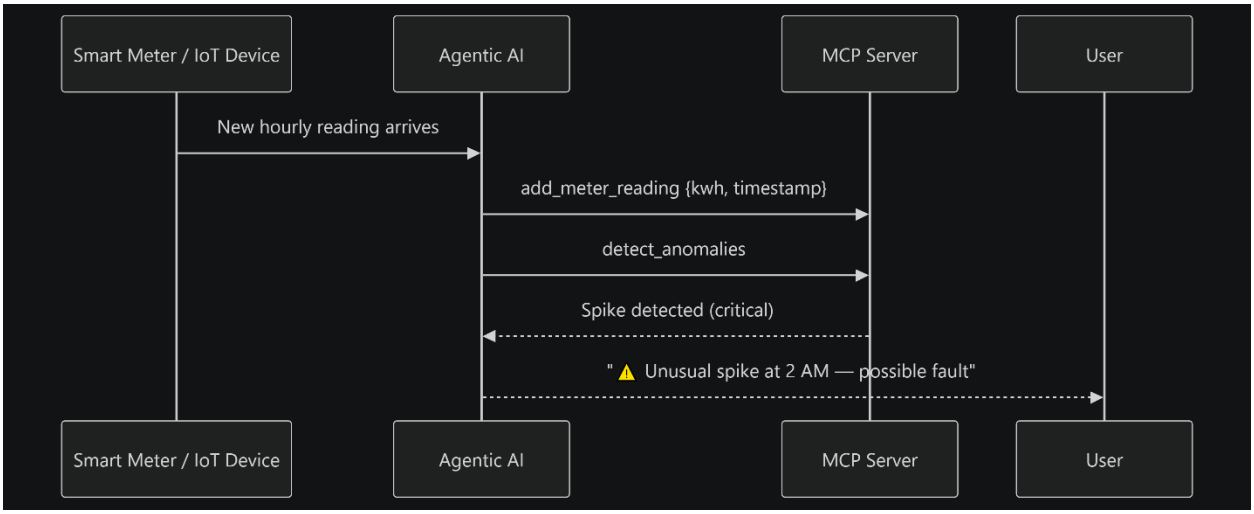


Figure 17: Smart Home Energy Advisor project screenshot 2

This is a **reactive agentic loop** — perceive → analyse → notify.

---

5. Decision-Making for Recommendations

The recommend_schedules tool doesn't just return data — it returns **actionable decisions**:

- Identifies which appliances are **shiftable** (e.g., washing machine, dishwasher, EV charger)
- Calculates the **optimal hours** to run them based on tariff rates
- Quantifies the **annual financial saving**

The agent can then **explain and justify** these recommendations in plain English, going beyond raw numbers.

---

6. Autonomous Anomaly Detection

detect_anomalies implements three detection strategies:

Table 27: Anomaly detection strategies

Detection Type	What the Agent Can Act On
Spike	Sudden kWh surge — possible fault or left-on appliance
Sustained High	Prolonged above-average usage — heating/cooling inefficiency
Overnight Drain	Unexpected consumption 11 PM–5 AM — standby waste or intrusion

An agentic AI can **proactively alert** the user without being asked — a hallmark of agentic behavior.

Summary

Table 28: Agentic capability usage

Agentic Capability	How It's Used Here
Autonomy	Agent picks the right tool without explicit instruction
Tool use	8 specialized tools exposed via MCP
Multi-step reasoning	Chains tools to answer complex questions
Context awareness	energy_advisor prompt injects live data into agent memory
Reactivity	Real-time reading ingestion triggers re-analysis
Proactive action	Anomaly detection enables unprompted alerts
Natural language interface	All interactions happen in plain English

The MCP architecture is the **bridge** between the AI's reasoning capabilities and the domain-specific energy data — making this a textbook example of an **agentic AI application**.

Novelty & Uniqueness of the Smart Home Energy Advisor

1. MCP as the AI Integration Layer (Not a REST API)

Most smart home energy tools expose a **REST API** that a developer must manually wire up. This project instead uses the **Model Context Protocol (MCP)** — meaning:

- The AI agent **self-discovers** all capabilities at runtime
- No frontend code, no HTTP routes, no API keys to wire up
- The server is **natively consumable by any MCP-compatible AI** (Bob, Claude Desktop, etc.)
- This is a fundamentally newer integration pattern — most energy management tools predate MCP entirely

---

2. Conversational Energy Intelligence — Not Just a Dashboard

Traditional energy apps show **charts and numbers**. This project enables:

"Why is my bill £40 higher this month?" "Should I charge my EV now or wait until midnight?"

The energy_advisor prompt **primes the AI with live data context**, turning raw meter readings into a **dialogue** — something no conventional energy dashboard does natively.

---

3. Three-Tier Anomaly Detection Engine

The detectAnomalies() function implements **three distinct detection strategies** in a single pure function:

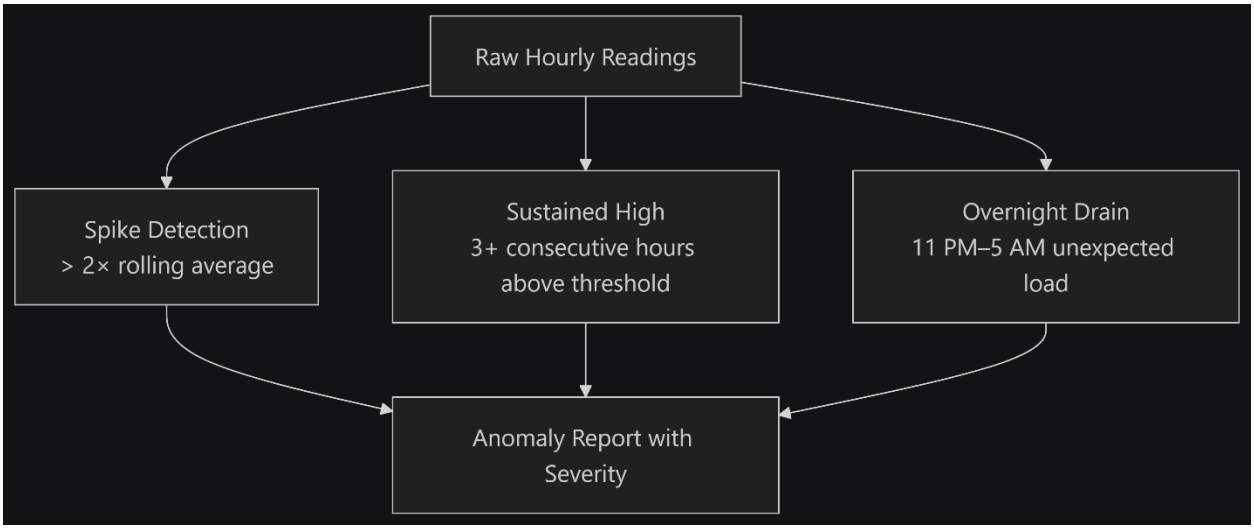


Figure 18: Smart Home Energy Advisor project screenshot 3

Most consumer energy tools only flag **billing anomalies** (month-over-month). This detects **intra-day operational faults** — a level of granularity typically found only in industrial SCADA systems.

---

4. Tariff-Aware Cost Optimisation

The recommendSchedules() function doesn't just say *"run appliances off-peak"* — it:

- Models **three tariff structures** (flat, time-of-use, dynamic/agile) via tariffs.ts
- Computes **actual pence-per-cycle savings** for each shiftable appliance
- Projects **annual savings in currency** — a concrete financial outcome
- Considers which appliances are physically shiftable vs. always-on

This is **demand-side energy management** logic usually found in utility-grade software, here made accessible via a single AI tool call.

---

5. Realistic Synthetic Demo Data with Domain Fidelity

The DEMO_METER ships with **720 hours (30 days) of synthesised readings** that deliberately encode:

- Cold-weather **heating spikes** (temperature-correlated kWh)
- **Morning/evening peak** human activity patterns
- An **EV charger running at peak hours** (the most expensive mistake a homeowner makes)

This means the server is **immediately explorable** without any real hardware — and the demo data is realistic enough to produce genuinely useful recommendations. Most demo/toy projects use flat or random data; this one uses domain-accurate simulation.

---

6. Pure Functional Analysis Engine

The entire energyAnalysis.ts layer is built as **pure functions** — no I/O, no state mutation, no side effects:

MeterData + Tariff → typed result object

This is architecturally unusual for "advisor" systems, which typically mix data fetching, business logic, and formatting in the same layer. The clean separation means:

- Every analysis function is **independently unit-testable**
- The same logic can power both the MCP tools **and** a future REST API or UI without modification
- The AI can call analysis functions **speculatively** (e.g., compare tariff plans) without corrupting state

---

7. Real-Time Push via add_meter_reading

The server accepts **live meter data pushes** through the add_meter_reading tool — enabling a reactive loop:

Smart Meter → add_meter_reading → detect_anomalies → AI alerts user

This means the same server that answers historical questions can also serve as a **real-time monitoring endpoint** — dual-mode operation in a single lightweight process.

---

8. Zero Infrastructure Footprint

Table 29: Typical energy platform versus this project

Typical Energy Platform	This Project
Cloud database	❌ In-memory Map
Authentication service	❌ Not needed (MCP handles trust)
Message broker / MQTT	❌ stdio transport
Web server / API gateway	❌ No HTTP layer
Frontend framework	❌ Single HTML file

The entire system runs as a **single Node.js process** consuming ~5 MB RAM — deployable on a Raspberry Pi sitting next to the smart meter. No cloud dependency, no subscription, no vendor lock-in.

Summary Table

Table 30: Project novelty dimensions

Dimension	What's Unique
Integration pattern	MCP-native — AI self-discovers tools, no manual wiring
Interaction model	Conversational Q&A, not just a static dashboard
Anomaly detection	Three-strategy intra-day fault detection (beyond billing alerts)
Optimisation	Tariff-aware, appliance-specific, currency-quantified savings
Demo data	Domain-accurate synthetic data (temperature, EV, peaks)
Architecture	Pure functional analysis engine — fully decoupled from I/O
Dual-mode	Historical analysis + real-time push in one server
Infrastructure	Zero cloud dependency — runs on a single process

The core novelty is the **combination**: taking utility-grade energy intelligence (tariff optimisation, anomaly detection, demand shifting) and delivering it through a

conversational AI interface with zero infrastructure overhead — a pairing that doesn't exist in the current smart home ecosystem.

Future Scope of Smart Home Energy Advisor

1. Real Hardware Integration

Currently the server ships with synthetic demo data. The next natural step is connecting to real smart meter APIs:

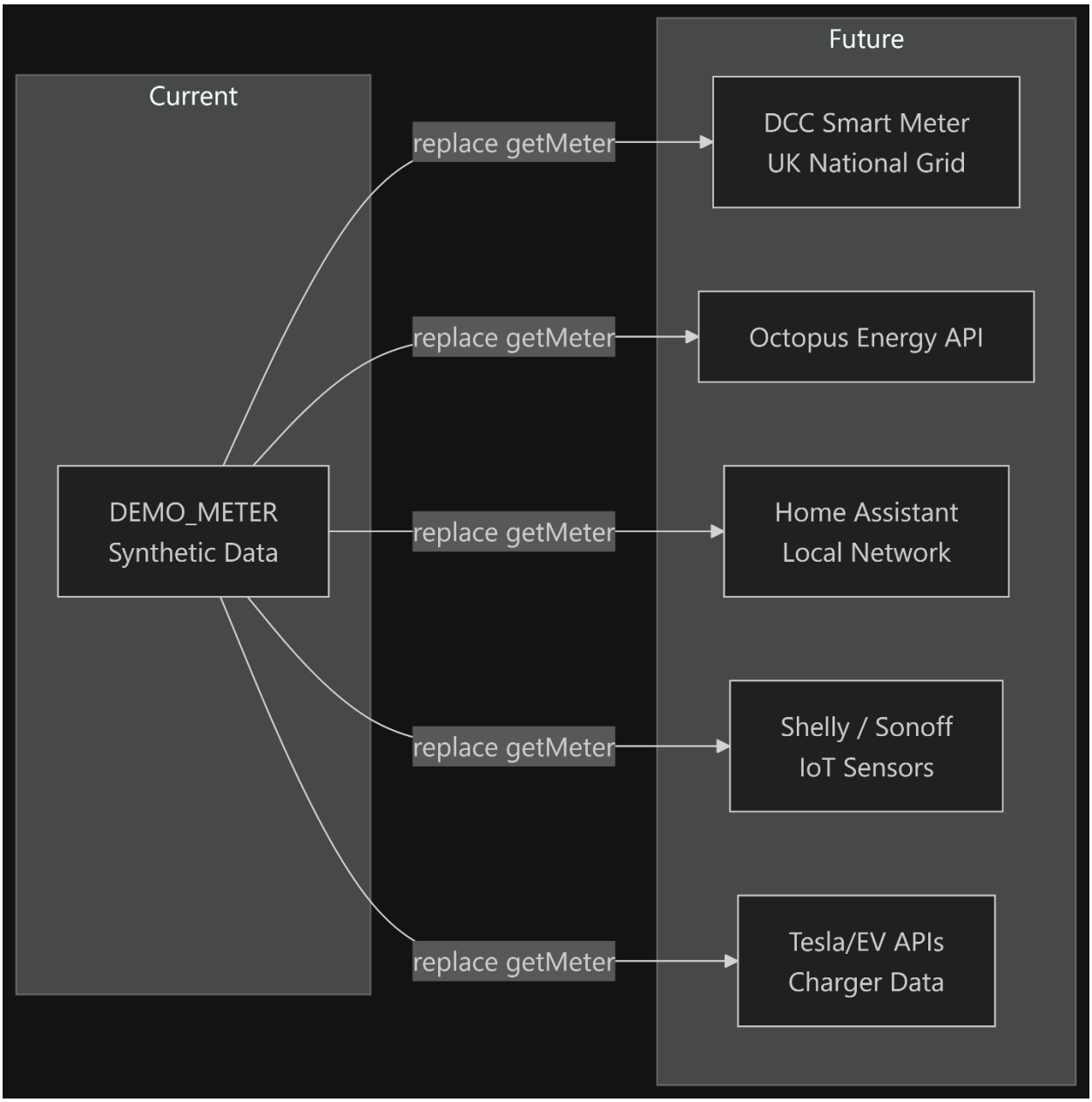


Figure 19: Smart Home Energy Advisor project screenshot 4

The getMeter() function is the **single swap point** — the rest of the architecture requires zero changes.

---

2. Persistent Storage Layer

The current in-memory Map loses all data on restart. A persistence layer would unlock:

Table 31: Storage options and use cases

Storage Option	Use Case
SQLite	Local single-home deployment, zero ops
PostgreSQL / TimescaleDB	Multi-home, time-series optimised queries
InfluxDB	IoT-grade time-series with nanosecond precision
Redis	Real-time caching of latest readings

The pure functional energyAnalysis.ts layer requires **no changes** — only the store layer swaps out.

---

3. Multi-Home & Multi-Tenant Support

The meter_id parameter already exists on every tool — the foundation for multi-tenancy is built in:

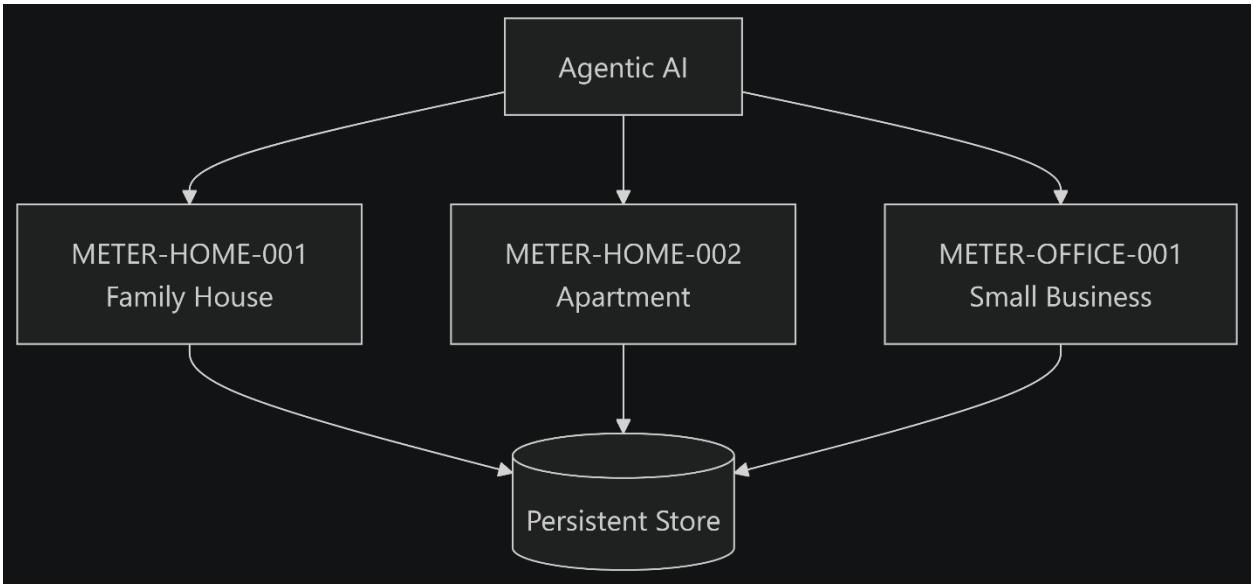


Figure 20: Smart Home Energy Advisor project screenshot 5

Future additions needed:

- User authentication & meter ownership
- Role-based access (owner vs. tenant vs. landlord)
- Aggregated cross-property reporting

---

4. Machine Learning & Predictive Analytics

Replace rule-based analysis with ML models:

Table 32: Rule-based versus ML-based analysis

Current (Rule-Based)	Future (ML-Based)
Spike = 2× rolling average	LSTM anomaly detection on time-series
Peak hour = max average	Clustering to find household behavioural patterns
Bill explanation = heuristics	Regression model trained on weather + occupancy
Schedule shift = cheapest tariff slot	Reinforcement learning for dynamic optimisation

The energyAnalysis.ts pure function interface stays the same — ML models slot in as drop-in replacements.

---

5. Carbon Footprint & Green Energy Tracking

Energy cost is only half the picture. Future scope includes:

- **Grid carbon intensity** integration (e.g., National Grid ESO Carbon Intensity API)
- Per-appliance **CO_2 emissions** tracking
- **Green tariff** comparison (renewable vs. fossil-fuel hours)
- Carbon budget alerts alongside cost alerts
- **Eco-score** per household — gamified sustainability metric

New MCP tools to add:

get_carbon_summary → CO_2 emitted this month

recommend_green_shifts → Run appliances during low-carbon grid hours

compare_tariff_carbon → Cost vs. carbon trade-off analysis

---

6. Solar & Battery Integration

For homes with solar panels or home batteries:

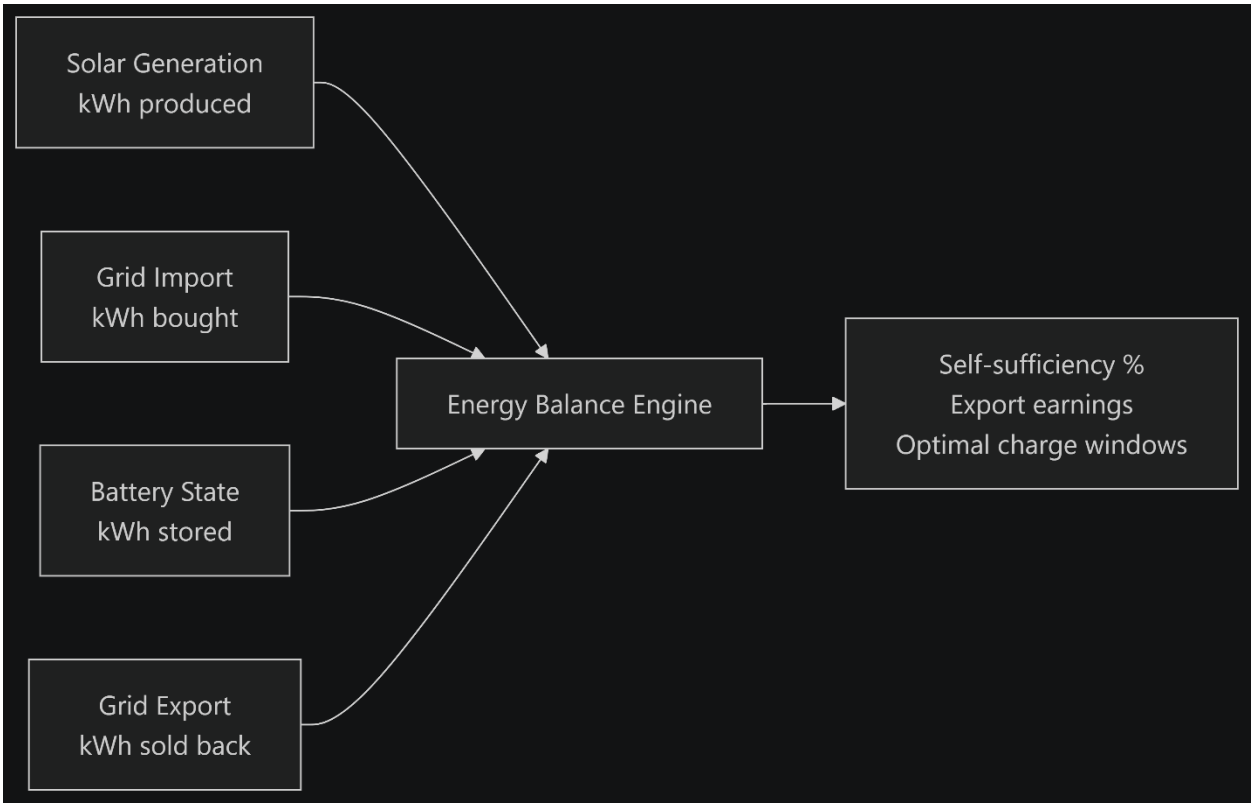


Figure 21: Smart Home Energy Advisor project screenshot 6

New tools:

- get_self_sufficiency_score — % of demand met by solar
- optimise_battery_charge — when to charge/discharge based on tariff + forecast
- forecast_solar_generation — weather-based generation prediction

---

7. Proactive AI Agent with Scheduled Monitoring

Currently the AI is **reactive** (answers questions). Future scope makes it **proactive**:

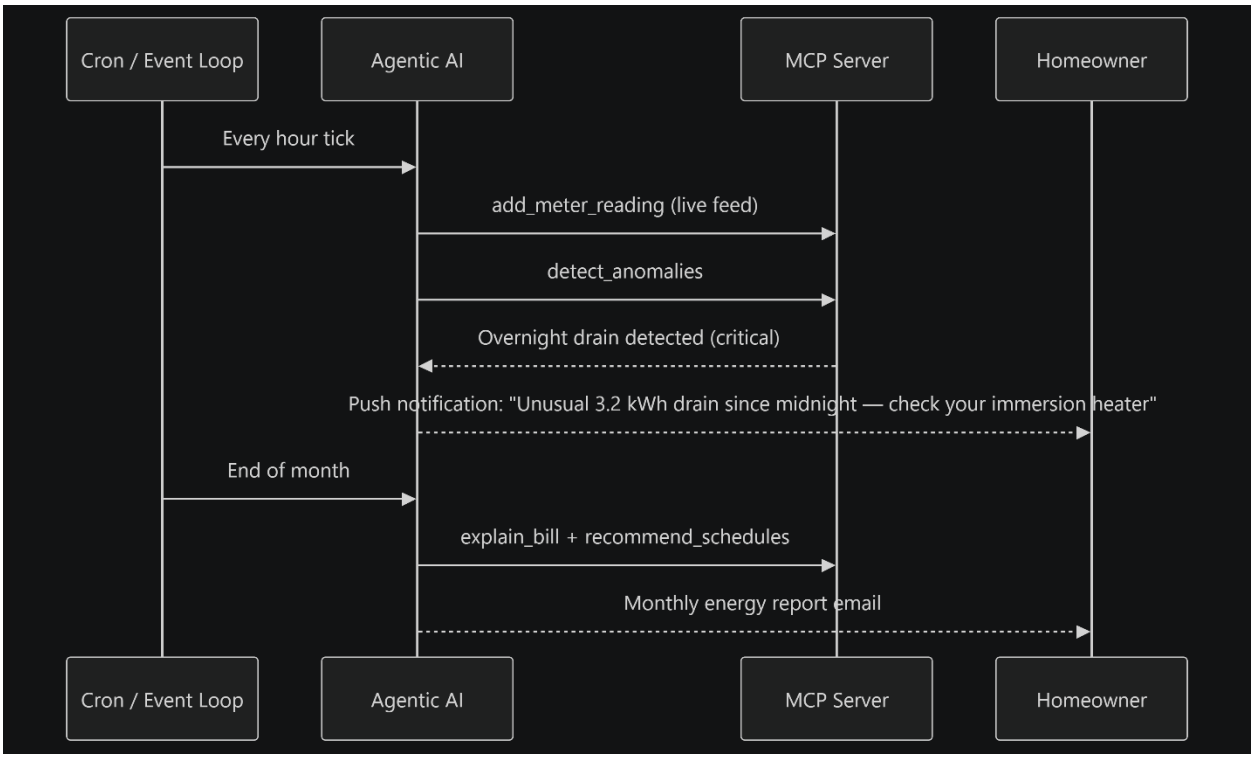


Figure 22: Smart Home Energy Advisor project screenshot 7

8. Voice & Conversational UI

The energy_advisor prompt already enables natural language — extend to:

- **Voice assistant integration** (Alexa, Google Home, Apple Siri via MCP bridge)
- **WhatsApp / Telegram bot** for push alerts and Q&A
- **Smart display** integration (Amazon Echo Show, Google Nest Hub)
- **Progressive Web App (PWA)** wrapping the dashboard.html into a mobile-first app

---

9. Expanded Tariff Engine

The current 3 tariffs (flat, TOU, dynamic) can expand to:

Table 33: Tariff types and descriptions

Tariff Type	Description

Tariff Type	Description
Octopus Agile	Real-time half-hourly pricing from wholesale market
Octopus Go	EV-specific overnight flat rate
Export tariffs	Smart Export Guarantee (SEG) rates for solar sellers
Economy 7/10	Legacy overnight cheap-rate tariffs
Demand response	Utility incentive programmes for load shifting
EV smart charging	Vehicle-to-grid (V2G) bidirectional tariffs

New tool: compare_tariffs — simulate switching tariff plans and show projected annual savings.

---

10. Home Automation Integration

Connect recommendations to **actual device control**:

The AI goes from **advisor** → **autonomous energy manager** — closing the loop between insight and action.

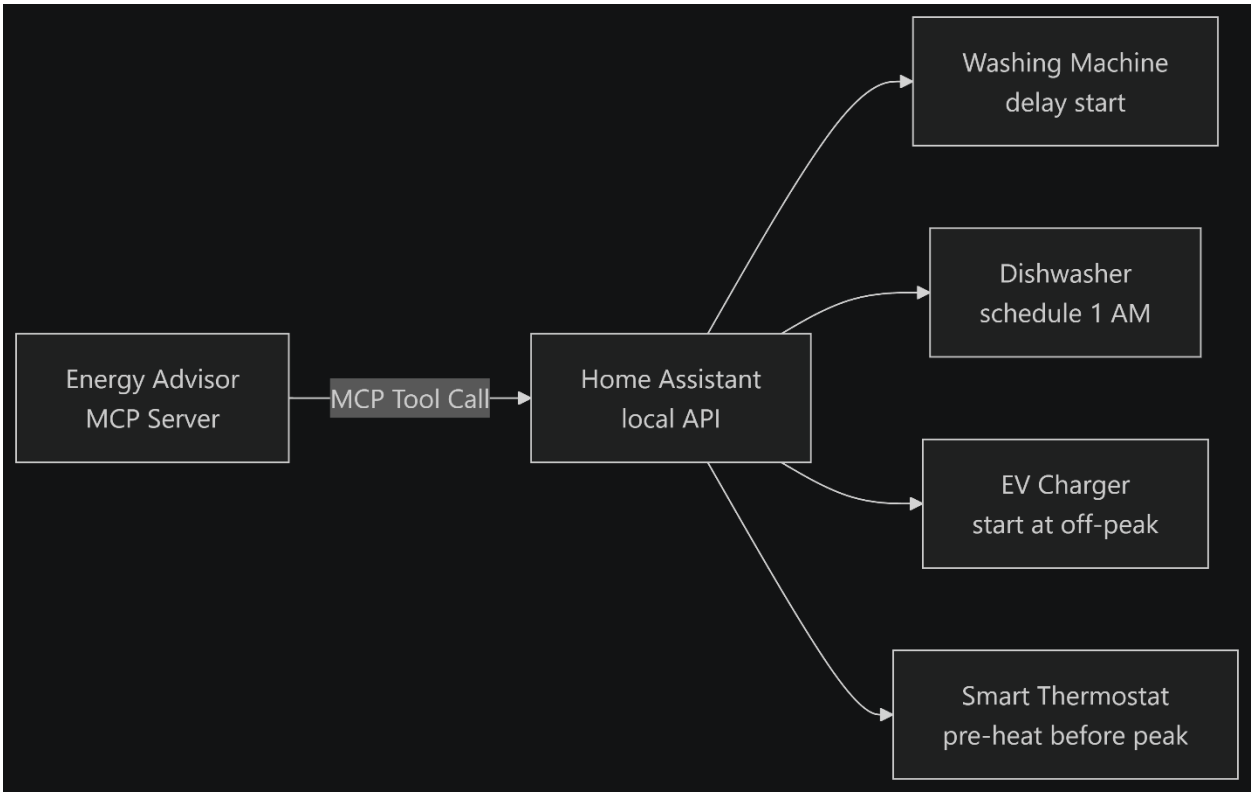


Figure 23: Multi-step agentic tool chaining

---

11. Demand Forecasting

Predict tomorrow's consumption to enable pre-emptive decisions:

- **Weather forecast** correlation (heating/cooling demand)
- **Occupancy prediction** (calendar integration — home vs. away)
- **Appliance usage patterns** (learned from historical cycles)
- Feed forecasts into recommend_schedules for even sharper optimisation

---

12. Community & Benchmarking

Compare a household against anonymised peers:

- *"Your home uses 23% more than similar 3-bedroom houses in your area"*
- Neighbourhood-level demand aggregation for utility load balancing

- Leaderboard / sustainability ranking for engaged users
- Privacy-preserving federated analytics (no raw data leaves the device)

Roadmap Summary

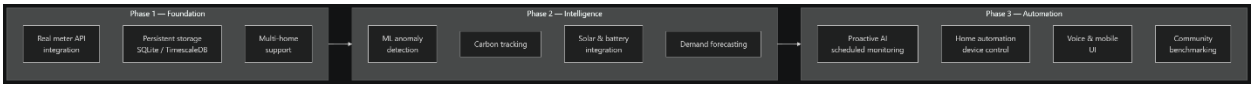


Figure 24: Real-time agentic loop